software

MDPI

Review

Machine Learning Techniques for Requirements Engineering: A Comprehensive Literature Review

António Miguel Rosado da Cruz [1,*] and Estrela Ferreira Cruz [1,2,*]

[1] ADiT-Lab, Polytechnic University of Viana do Castelo, 4900-347 Viana do Castelo, Portugal
[2] Algoritmi Research Centre, Escola de Engenharia, Universidade do Minho, 4800-058 Guimarães, Portugal
* Correspondence: miguel.cruz@estg.ipvc.pt (A.M.R.d.C.); estrela.cruz@estg.ipvc.pt (E.F.C.)

Abstract

Software requirements engineering is one of the most critical and time-consuming phases of the software-development process. The lack of communication with stakeholders and the use of natural language for communicating leads to misunderstanding and misidentification of requirements or the creation of ambiguous requirements, which can jeopardize all subsequent steps in the software-development process and can compromise the quality of the final software product. Natural Language Processing (NLP) is an old area of research; however, it is currently undergoing strong and very positive impacts with recent advances in the area of Machine Learning (ML), namely with the emergence of Deep Learning and, more recently, with the so-called transformer models such as BERT and GPT. Software requirements engineering is also being strongly affected by the entire evolution of ML and other areas of Artificial Intelligence (AI). In this article we conduct a systematic review on how AI, ML and NLP are being used in the various stages of requirements engineering, including requirements elicitation, specification, classification, prioritization, requirements management, requirements traceability, etc. Furthermore, we identify which algorithms are most used in each of these stages, uncover challenges and open problems and suggest future research directions.

Keywords: artificial intelligence; Machine Learning; natural language processing; requirements engineering; software engineering

Academic Editors: Vincenzo Gervasi and Mirko Viroli

Received: 29 March 2025
Revised: 21 June 2025
Accepted: 23 June 2025
Published: 28 June 2025

Citation: Rosado da Cruz, A.M.; Cruz, E.F. Machine Learning Techniques for Requirements Engineering: A Comprehensive Literature Review. *Software* **2025**, *4*, 14. https://doi.org/10.3390/software4030014

1. Introduction

Requirements engineering is one of the most critical phases of the Software development life-cycle (SDLC), or software-development process, since an error in this phase can generate rework, additional costs and even compromise the success of the entire project. However, requirements engineering faces many challenges such as the difficulty in obtaining the correct and unambiguous information from stakeholders, the complexity in identifying all the actors involved, the software features they may access, domain-relevant entities, the impact of a requirement's change, among many others. These difficulties arise from the ambiguity of natural language and misalignment of thought.

After collecting the requirements information, it is necessary to document, in an organized manner, all the information collected; that is, the Software Requirement Specification (SRS) must be created. SRS is a document that serves as a working basis for the rest of the software-development process, including testing [1]. Ambiguous or incomplete descriptions of requirements can be problematic for the whole software project or solution.

Requirements can be classified into Functional Requirements (FR) and Non-Functional Requirements (NFR). FR describes what the system should do, that is, its functionalities and expected behaviors. NFR specifies how the system should operate, encompassing aspects such as performance, scalability, security and usability.

Requirements Engineering (RE) tasks, being an important part of the software-engineering discipline, have long been "promoted" to their own discipline. Requirements engineering deals with the elicitation, analysis, formalization, specification, documentation and validation of requirements, among other activities, in any software or system project. In this context, a system will always involve software, but may also have non-software components, such as sensors and actuators, in addition, of course, to different types of human users.

Artificial Intelligence (AI), despite having a long history, has undergone significant developments in recent times, especially in its Machine Learning aspect. AI can be defined as the use of technologies to create machines that are capable of imitating cognitive functions associated with human intelligence, such as the ability to see, understand and respond to spoken or written language, analyze data, make recommendations and much more. AI typically uses a set of technologies implemented to enable them to learn, reason and act together to solve complex problems [2]. Machine Learning (ML) is the area of AI that allows a machine, or a system, to learn based on experience. To do this, ML uses algorithms to analyze large volumes of data, learn and make decisions based on that analysis.

The growing number and variety of algorithms in ML have played a key role in advancing the field. Among these, Deep Learning algorithms have been particularly instrumental in driving major breakthroughs in natural language processing. Natural Language Processing (NLP) itself has a rich history, dating back to the 1940s with early efforts in automatic language translation. Its development has mirrored the broader evolution of AI, moving through symbolic, statistical and neural network-based approaches. In recent years, Transformer-based models such as BERT and GPT, which are Deep Learning Architectures capable of handling vast amounts of data and capturing intricate patterns, have led to remarkable progress in the field of NLP [3].

Requirements collection, analysis and processing are closely related to NLP, so it is natural that the evolution felt in the area of NLP is also felt in the area of RE. However, requirements engineering involves other tasks, such as requirements classification and prioritization, etc., to which more traditional ML algorithms, such as classification and regression algorithms, can also contribute positively.

In this article, we will carry out a comprehensive literature review of the ML strategies used in RE. We leverage all the ML algorithms currently used in each of the stages of the requirements engineering process. After summarizing the main requirements engineering activities and presenting some concepts and the evolution of ML algorithms, this article reviews the AI techniques used for RE. The analysis developed seeks to identify the techniques described in the scientific literature in the last five years and the first months of 2024, between 2019 and 2024, to support requirements engineering tasks or solve problems that occur during these tasks.

1.1. Requirements Engineering Tasks and Issues

This subsection provides a summary of the main tasks of requirements engineering, and problems that exist when these tasks are developed by humans in large projects, or with a large number of requirements, and problems that arise when we try to automate these tasks, even partially.

Any systems project begins with a necessity from its users, typically a business need [4]. A systems project team needs to identify, contextualize and understand this necessity, before

being able to formalize, prioritize and document requirements. This process involves some requirements engineering activities and tasks [4–6]:

- Inception: This first activity involves, basically, the identification of a necessity, which will trigger a new project to develop a system capable of provisioning the necessity.
- Requirements Elicitation: This involves identifying the sources of requirements, and gathering requirements using various available techniques, such as interviews, observation of the environment and work processes that the system will support, etc.
- Requirements Elaboration: This entails an analysis of the previously gathered requirements, their contextualization in the problem domain, and the identification of ambiguous, contradictory or meaningless requirements. It also involves the classification of requirements into functional and non-functional, as well as in the latter case their classification into an NFR category. In Figure 1, the types and dimensions of requirements are illustrated, leveraging FR and NFR.

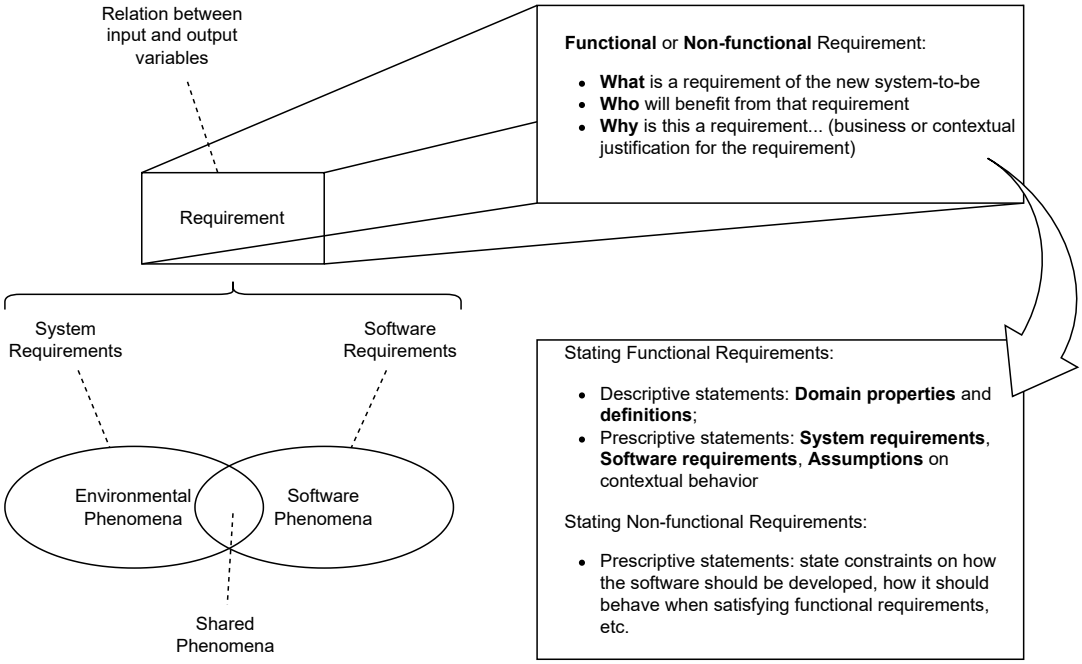

Figure 1. Types and dimensions of requirements (the figure aggregates and summarizes information taken from [4,6] and reflects the authors' viewpoint).

- Requirements Negotiation: In this phase, candidate requirements, resulting from the previous activities, are negotiated, regulating divergences and adopting prioritization techniques [4].
- Requirements Documentation: Requirements documents, namely SRS documents, serve as the main reference for the subsequent software-engineering phases. These documents must display a set of requirements with a formalized structure, and the respective quality and verifiability criteria. At this stage, requirements are typically organized according to two perspectives: user requirements, which describe users' needs; and, system requirements, which describe how the system should behave in different situations [4]. Both these perspectives may include FR and NFR.
- Requirements Validation: This activity includes examining the documented requirements and evaluating if they describe the system desired by the client. This may involve technical inspections and reviews and its main goal is to prevent defects in

requirements from propagating to the following phases of the SDLC. Errors detected in this phase have much lower costs than errors detected in subsequent phases [4].

- Requirements Management: This activity runs throughout the whole system-development process. Its main goal is to manage requirements and their changes. Requirements traceability is a tool for keeping track of requirements aspects. For example, it may be used for tracing requirements change (each requirement is linked back to its previous version), requirements dependability (each requirement is linked to the requirements on which it depends), system features and requirements (each feature is linked to a set of logically related requirements).

1.2. Main Aim and Research Questions

Previous literature reviews have explored the application of AI in requirements engineering [7,8], and thus have been identified in this study's search results. Previous reviews from the last five years, which are analyzed in Section 4, either take a broad approach, examining the use of AI across multiple phases of the software-development lifecycle, from modeling and design to testing [9–12]; focus on specific requirements engineering tasks, such as prioritization [13,14], classification [15–17], specification [1] or traceability [18]; address ambiguity in the identification and specification of requirements [19]; or target recurring needs across projects, such as using AI to generate requirements that have to be aligned with the General Data Protection Regulation (GDPR) [20]. Most of them analyze studies from an earlier time period than the review presented in this article.

What we could not find in previews reviews was a clear identification of the RE activities that can most benefit from the use of AI techniques, which techniques are most used for each RE activity or group of activities, and which ones achieve the best results in each RE activity or group of activities.

The objective of this research is, then, to answer the following research questions:

RQ1 Which Requirements Engineering activities take advantage of the use of Artificial Intelligence techniques?

RQ2 Which Artificial Intelligence techniques are most used in each Requirements Engineering activity?

RQ3 Which Artificial Intelligence techniques have the best results in each Requirements Engineering activity?

1.3. Structure of the Article

Te rest of this article is organized as follows. The next section presents the methodology used in the literature review. Section 3 summarizes the Machine Learning techniques, organizing them into different categories and methods. In Section 4, a synthesis of previous literature reviews in this area is carried out. The results obtained in this work's literature review are detailed in Section 5. Section 6 presents a discussion on the results obtained, and conclusions are presented in Section 7.

2. Materials and Methods

The state-of-the-art literature of the last five years on AI approaches to RE tasks and issues is reviewed in this article. To answer the previously defined research questions, a search query was defined and carried out on Scopus and on Web of Science (WoS) on 31 May 2024, with our research work having been conducted between May and October 2024.

The research methodology used for this study follows the protocol defined in [21]. The search strategy (databases, search query, criteria for inclusion or exclusion) and the full screening method used for the literature review are depicted in Figure 2.

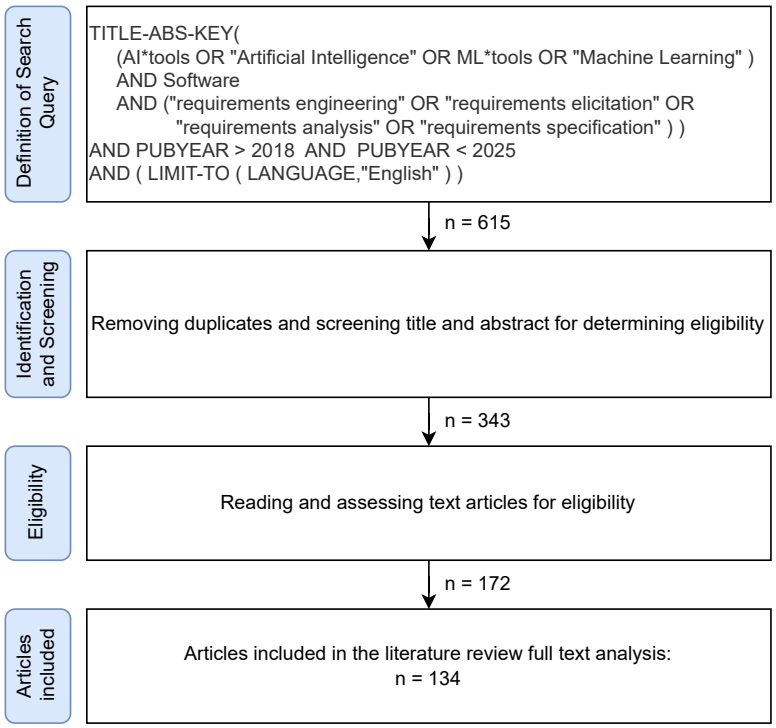

Figure 2. Prisma flow diagram for the methodology used in this survey.

The search query was defined to find indexed journal articles and conference papers, published between 2019 and 2024, on the usage of AI-based techniques and technologies to help with activities of the requirements engineering process. After removing duplicates, the articles were further selected by screening through title and abstract. In this phase, articles on software-engineering techniques for AI applications were removed, as the goal is to address AI techniques for software engineering focusing on requirements engineering. In this phase, previous surveys and literature reviews were also removed. Further analysis allowed us to eliminate some more articles, which were not also aligned with the defined goal. Finally, 134 articles were selected for a more thorough analysis.

3. Machine Learning Techniques

In this section, the main ML concepts and algorithms are summarized.

For decades, and especially in recent years, AI and its subfields of ML and NLP have received major investments worldwide, which has driven its rapid evolution. This evolution is reflected in the daily lives of each person and in almost all areas of business, in industry, commerce, finance, etc. [22]. Software development, including the areas of requirements engineering, is also positively influenced by this evolution.

Machine Learning is a branch of AI that allows computers to detect patterns in data and, based on that, make decisions to solve problems for which they were not explicitly programmed [23]. To do this, it is necessary to use complex algorithms that can be grouped into four broad categories, according to the way they learn from data [22–24]:

- Supervised Learning—the model learns from labeled data, where input features are already associated with the correct outputs. These types of algorithms can be used in predictions such as price prediction, correct/incorrect classification and medical diagnosis. Within this group, we can further separate the algorithms between classification algorithms (used to classify yes/no) and regression algorithms (used to predict

continuous values, such as prices or temperatures). In this group, we have algorithms such as Linear Regression, Logistic Regression, Decision Tree (DT), Random Forest (RF), Support Vector Machines (SVM), Multi-Layer Perceptron (MLP), Artificial Neural Networks (ANN)s, K-Nearest Neighbors (KNN), Gradient Boosting (XGBoost, LightGBM, CatBoost), etc.

- Unsupervised Learning—the model learns without labeled data, identifying hidden patterns in the data. These algorithms can be used for clustering, anomaly detection, etc. In this group, there are algorithms such as K-Means, which groups data into K clusters; Hierarchical Clustering, which creates a hierarchical structure of clusters; or DBSCAN, which identifies dense groups of points, useful for unstructured data.
- Reinforcement Learning—in this group, algorithms learn through trial and error, receiving rewards or punishments. This type of algorithm is used in games, robotics and optimization of financial strategies. This group includes algorithms such as Q-Learning, a table-based algorithm for finding the best action, Deep Q-Networks (DQN), which uses neural networks for Deep Learning; Proximal Policy Optimization (PPO), an advanced algorithm used by OpenAI, etc.
- Deep Learning—this involves algorithms that use artificial neural networks with multiple layers to learn complex representations of data. These algorithms are especially used in image recognition, natural language processing and speech and audio processing [24]. In this group, there are algoritms such as ANN, Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU). Pre-trained Language Models (PLM) algorithms, such as Bidirectional Encoder Representations from Transformers (BERT) and Generative Pre-trained Transformer (GPT), also belong to this group.

ML algorithms can be grouped into different categories depending on how they learn, the type of architecture they use or the type of problem they solve. Figure 3 groups the most common ML algorithms based on the type of learning. The most commonly used algorithms in NLP are represented in bold letters in the figure.

The strong advances in ML are having a very positive impact in NLP, having revolutionized the area. Natural language is complex, subject to many rules and, at the same time, can be ambiguous, can be context-dependent and can involve inferences and intentions, such as irony, sarcasm and metaphors. Furthermore, natural language itself is constantly changing and evolving. This makes processing human language a difficult task.

NLP has a long history. Since 1980, researchers have been working to automate requirements engineering tasks using NLP techniques [25]. NLP uses algorithms to analyze, understand and generate human language [25].

NLP includes tasks like document classification, paraphrase identification, text similarity identification, summarization, translation, etc. [24]. To do that, most of the NLP models involve steps like tokenization, where the text is broken down into words, and representation, where these words are represented in the form of vectors or n-grams (used to analyze sequences of n words to understand the context). Deep Learning (DL) algorithms have been the most successful ones in performing these tasks, which is why they are the most used in NLP [24]. DL has paved the way for the emergence of PLM. PLM such as BERT and GPT have given NLP a huge boost [3].

Thus, NLP may use models such as LSTM, for processing long texts, machine translation, transformers, which are models that allow parallelization and deeper contextual understanding, among other models. The transformer model is a natural language processing model proposed by Google in 2017, which can include algorithms such as BERT, GPT, Text-To-Text Transfer Transformer (T5) and others [26].

NLP also makes use of more traditional algorithms such as Naïve Bayes (NB), for Text Classification; SVM, for Sentiment Analysis and Text Categorization; RF and DTs, for classification and entity extraction; among others.

Most solutions do not just use one algorithm, but use several algorithms that complement each other and, together, present a better solution.

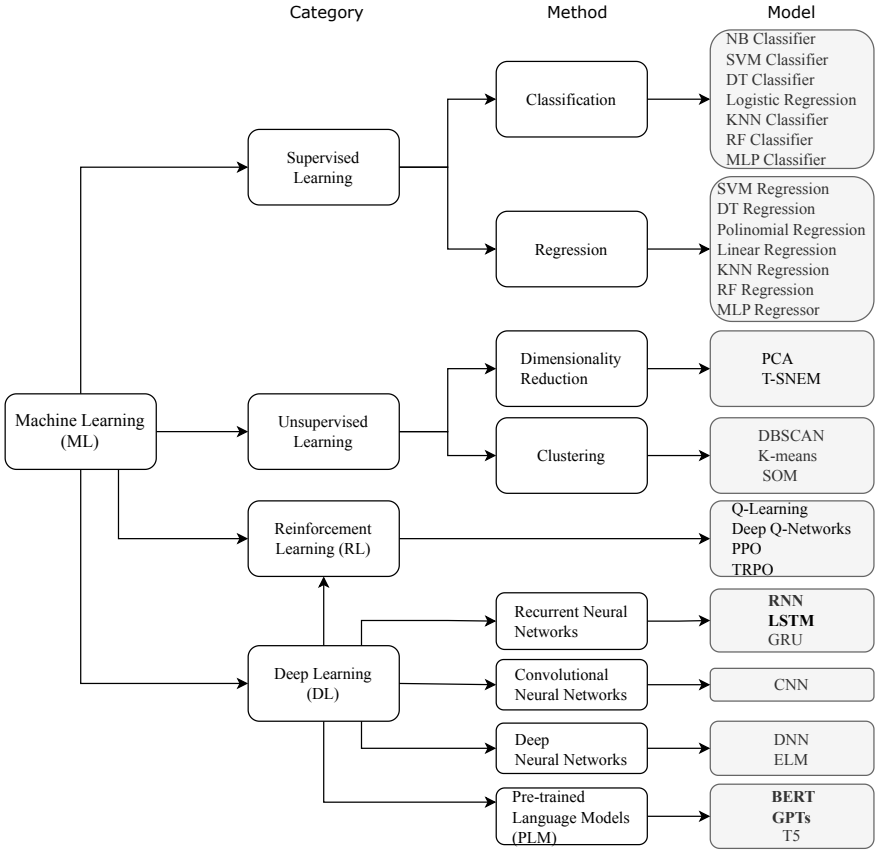

Figure 3. Summary of Machine Learning algorithms (adapted from [23]).

4. Previous Literature Reviews

This section analyzes previous literature reviews that have explored the application of AI in requirements engineering, and have been identified in this study's search results.

Prioritizing requirements is a critical task, particularly in large projects where the volume can make manual handling tedious. In [13], the authors conduct a systematic review of studies using AI tools and algorithms for requirement prioritization. They also examine current approaches involving ML, highlighting its advantages over other AI techniques.

In [18], the authors review studies that apply ML techniques to requirements traceability. They analyze which ML methods and datasets are most commonly used in requirements analysis and traceability, concluding that different algorithms are favored at different stages of the process. The traceability process is divided into three main stages: preprocessing, link generation and link refinement. In the preprocessing stage, Word2vec and Doc2vec are the most commonly used algorithms. For link generation, predicting connections and enabling traceability, RF, DT and NB are the most frequently applied. Regarding datasets, the authors note that around 50% of the reviewed studies do not specify their datasets, and among those that do, open source datasets are used more often than closed source ones.

In [9], the authors review studies from 2015 to 2021 that apply ML techniques across all stages of the software-development process, from requirements elicitation to design, implementation, testing and maintenance. They conclude that ML algorithms are beneficial at every stage, with certain algorithms better suited to specific phases. For the requirements elicitation phase, the most commonly used are supervised ML algorithms such as SVM, NB, DT, KNN and RF.

In [27], the authors conduct a systematic review to identify key challenges in requirements engineering and the techniques used to address them. They highlight poor communication among stakeholders and limited understanding of the problem as major issues, often leading to ambiguous, incomplete, inconsistent or incorrect requirements. Other challenges include requirement prioritization and maintaining documentation. The authors recognize artificial intelligence as a highly promising approach for addressing these recurring problems.

Many authors propose tools and methods to generate software models from information in requirements specifications, typically written in natural language [10]. In [28], the authors present a systematic literature review of approaches for transforming natural language into Unified Modeling Language (UML) diagrams. These approaches cover nearly all types of UML diagrams, including class, use case, object, sequence and activity diagrams. Many solutions are based on heuristic rules, which the authors identify as a promising direction for future research.

The ability to extract model elements from requirements increases the importance of requirements engineering, as it can enable automation of subsequent SDLC steps. In [10], the authors review tools and methods for generating UML models, particularly class diagrams, from software requirements. They conclude that while many approaches can identify classes (sometimes redundantly), most struggle to detect relationships between them and are often complex with notable limitations.

In [16], the authors survey the state-of-the-art articles that use ANN to classify software requirements. They conclude that the most used algorithms in requirements classification are NB, SVM and ANN, and PROMISE was the most popular database for those studies. Some of the studies classify functional requirements, but most of them focus on non-functional requirements, namely security and usability requirements.

NLP is gaining significant attention and driving notable technological progress. Since software requirements are often written in natural language, they are prone to ambiguity, one of the major challenges in requirements specification, potentially affecting later development stages. To address this, NLP-based tools are being applied in RE to reduce ambiguity. In [19], the authors analyze and compare automated and semi-automated disambiguation tools, concluding that while none fully eliminate ambiguity, some show promising results.

In [7], the authors examine ML techniques for automating RE activities. They conclude that automating analysis tasks can significantly reduce both cost and time. However, the study also finds a lack of clear standards or guidelines for selecting suitable ML and NLP techniques, and that most approaches combine several ML techniques to achieve better results. Similarly, there is no consensus on how to choose appropriate datasets.

The study in [14] presents a systematic review of ML algorithms used for requirements prioritization, aiming to identify the most efficient ones in classifying and prioritizing requirements. The authors conclude that the most commonly used algorithms are SVM, DT, KNN, NB, Linear Regression (LR) and Multinomial Naïve Bayes (MNB), in that order.

The study in [29] offers a detailed analysis of the representations used in NLP as input for ML techniques. The way requirements are represented significantly impacts the performance of NLP algorithms. The authors survey the state of the art across various RE stages and note major advancements in recent years, particularly with advanced embedding

techniques that have enhanced tasks like requirements analysis and extraction. However, traditional representations based on lexical and syntactic features are still widely used in tasks such as modeling and quality tasks at the syntax level [29].

In [8], the authors present a state-of-the-art review of articles published between 2015 and 2021 on the use of AI in RE. They conclude that applying NLP techniques and supervised learning to requirements documentation, particularly in the elicitation, specification and validation stages, is a growing trend [8].

In [12], the study takes a broader view, examining general trends in the use of AI techniques across all stages of software engineering. The authors conclude that RE and testing are the most researched stages for AI applications, though other stages, such as software design, also show strong potential.

Classifying requirements into categories can be tedious, time-consuming and error-prone. In [17], the authors review the literature on the use of ML techniques for software requirements classification. Their findings show that the most commonly used algorithms are NB, DT and NLP.

The systematic review presented in [20] focuses on how NLP can help automate the identification of requirements for GDPR compliance. Created by the European Union (EU) to protect personal data and the privacy of European citizens, the GDPR requires all software operating in the EU to meet specific requirements. The study identifies several ways NLP and NLP-based ML techniques can support RE tasks to help ensure compliance.

The study in [30] presents a systematic review of tools and techniques used for requirements validation. The authors classify validation methods into six categories: prototyping, inspection, knowledge-oriented, test-oriented, modeling and evaluation, and formal models. They find that knowledge-oriented approaches, such as ML methods, are the most frequently cited. The study also highlights the need to standardize requirements quality characteristics, with correctness, completeness, consistency and ambiguity being the most commonly mentioned. Other noted attributes include understandability, reusability, unexpected dependencies, variability and testability [30].

In [31], the authors examine the use of NLP techniques in Crowd-Based Requirements Engineering (CrowdRE), focusing on the analysis of online user feedback about software products. They conclude that ML is commonly applied in this context, with NB being the most frequently used algorithm, followed by SVM.

In [32], the authors study and classify the types of ML algorithms used to identify software requirements on Stack Overflow. They find that Latent Dirichlet Allocation (LDA) combined with Bag of Words (BoW) is the most commonly used approach. However, they also note that ML algorithms still face challenges in accurately identifying requirements.

5. Results

For the purpose of this study, the surveyed literature has been categorized into the following five categories of RE tasks:

- Classification of requirements according to their functional/non-functional nature;
- Supporting requirements elicitation;
- Improving the quality of requirements and software;
- Extracting knowledge from requirements;
- Supporting requirements management and validation, and project management.

This section presents the results of the developed literature review. In Section 5.1, the five categories of RE tasks are further explained. In Section 5.2, references that address each of the task categories are examined. The aim is to later, in Section 6, be able to identify common ML approaches and techniques used in the development of the task in question.

5.1. RE Categories of Tasks

The five identified RE categories of tasks, used to categorize the surveyed literature, are further explained in the following subsections.

5.1.1. Classification of Requirements According to Their Functional/Non-Functional Nature

A requirement expresses a user need or some constraint imposed on a system. According to the IEEE 610.12-1990 standard, a requirement is a condition or capability that must be verified or possessed by a system to satisfy a contract, standard or specification; and a documented representation of a condition or capacity, within the scope of the previous point [4].

Each requirement must be written in a form that:

- Is clear, unambiguous and easy to interpret;
- Expresses objective intentions and not subjective opinions.

Requirements may be divided into functional (FRs) and non-functional (NFRs) requirements. FRs express functionalities that the system should exhibit to its users, whilst NFRs impose restrictions on the system as a whole.

NFRs may be further classified into several subcategories, such as [4]:

- Appearance, which is about the visual aspect of the system's graphical user interface;
- Usability or User Experience (XP), which has to do with the system's ease of use and the friendliness of the user experience;
- Performance, related to characteristics of speed, storage capacity, ability to scale to greater numbers of simultaneous users, among other aspects;
- Security, having to do with authentication and authorization access to the system and to the data, data protection and integrity, etc.;
- Legal, namely standards, laws and rules that apply to the system or to its domain of application.

Many other NFR categories may be considered, and these are easily found in any software-engineering book, such as [4,5].

5.1.2. Supporting Requirements Elicitation

Requirements elicitation involves identifying the sources for requirements and the actual gathering of the requirements to form a set of candidate requirements. This can be done with a range of requirements elicitation techniques. This range includes techniques such as interviews, focus groups, surveys, introspection and observation of workers while doing their work to understand the way they work and where and how the system can improve it [4,5].

5.1.3. Improving the Quality of Requirements and Software

Elicited requirements are considered as candidate requirements because they lack further analysis, elaboration, negotiation and acceptance by the stakeholders. For this, a requirements engineer must further elaborate the requirements to ensure that there are no [4,5]:

- Contradictory requirements;
- Ambiguous requirements;
- Incoherent or senseless requirements;
- Complex requirements or requirements that need to be further divided into several requirements.

After elaboration and negotiation, requirements accepted by the stakeholders form the body of the SRS document.

5.1.4. Extracting Knowledge from Requirements

After establishing a set of accepted stable requirements, the system-analysis modeling follows. This RE activity yields a set of models, each representing a different perspective of the system being conceptualized and, together, comprising a technology-free model of the system. These different model perspectives are based on the requirements in the SRS. Examples of knowledge that may be extracted from the requirements are, among others, the following:

- Rewriting requirements in a standard form;
- System features;
- Types of system users;
- System structural entities;
- Dependency between requirements;
- Related requirements, enabling requirements traceability.

5.1.5. Supporting Requirements Management and Validation, and Project Management

Requirements management is an activity that spans the entire SDLC and includes tasks such as assessing the impact of changing requirements.

Requirements validation helps ensure that the established requirements define the system desired by the client [4].

5.2. ML Techniques Used in RE Tasks

The reviewed literature items were categorized into the five RE task categories presented in the previous subsection. The specific references found in the literature, for each category, have been organized in tabular form and are depicted in Table 1. Some references may appear in more than one category, in the cases they address several RE tasks.

Table 1. Categorization of the reviewed literature, according to the requirements engineering phase and activity addressed.

RE Category	RE Activity	References
Classification of Requirements according to their Functional/ Non-Functional nature	Binary FR/NFR Classification, and ternary Classif. FR/NFR/Non-Req	[33–58]
	Classifying NFR in further subcategories, and Classifying ASFR	[35,38,40,41,43,45,47–49,52,57,59–61]
Supporting Requirements Elicitation	Categorize and Classify Business Rules (as a source for RE)	[62]
	Predicting/recommending Techniques for Req. elicitation	[63]
	Generating questions for Requirements elicitation	[56,64]
	Identifying/Generating new Requirements from existing Requirements or from User Feedback; Identifying ASFR; Classifying User Feedback	[39,51,55,65–79]

Table 1. *Cont.*

RE Category	RE Activity	References
Improving Quality of Requirements and Software	Requirements itemization, simplification, disambiguation; Detecting incompleteness, ambiguity, inaccuracy, implicit requirements, semantic similarity, and other risks (smells) in Requirements Specification; Coping with ambiguity through the use of controlled vocabulary and Ontologies	[39,58,80–93]
	Verification of Quality Characteristics in NFR (e.g., usability, UX, security, explainability) and user feedback; Ensuring security of Requirements; GDPR; Assessing the SRS quality by ISO 25010; Test case generation/Automate the quality checking/analysis of a Req./user story	[94–100]
	Identify potential effects on Sustainability; Assess Transparency and Sustainability as NFR	[101–103]
	Verification of pre-/post-conditions of Requirements; Requirements to Test Cases Traceability; Predicting probability of defects using design-level attributes	[96,104–110]
	Classification of requirements in two classes: "Integration Test" and "Software Test" using ML approaches	[111]
Extracting Knowledge from Requirements	Requirements Formalization; Extracting/Associating Features (Feature Extraction) or Model Elements from/to Requirements; Extract domain vocabulary from requirements for Feature Modeling or ontology construction	[43,58,81,83,112–134]
	Detecting or Extracting Requirements Dependencies/Requirements Traceability (forward and backward)	[105,120,121,135–143]
Supporting Requirements Management and Validation, and Project Management	Requirements Prioritization	[77,144–149]
	Allocating requirements to software versions based on development time, priority, and dependencies; Predicting whether a software feature will be completed within its planned iteration	[150–152]
	Project Management Risks Assessment/Req. Change Impact analysis on other requirements and on planned test cases	[153–155]

In this section, the main ML techniques used in the reviewed literature, for each of the RE categories of tasks, are analyzed.

5.2.1. Classification of Requirements According to Their Functional/Non-Functional Nature

Several ML techniques may be used for classifying requirements, from natural language text in SRS documents, according to their Functional/Non-Functional nature.

Requirements classification pipelines typically begin by preprocessing SRS text, using tokenization, lemmatization and feature extraction (e.g., BoW, embeddings), to convert natural language into numerical vectors. These vectors feed into ML classifiers that perform either binary (functional vs. non-functional), ternary (adding "non-requirement") or more granular categorization of NFR subtypes and Architecturally Significant Functional Requirements (ASFR).

In Table 2, the main approaches for the early NLP phase, and the main ML-based approaches to the classification of requirements according to their Functional/Non-Functional nature, are presented. As depicted in the table, several techniques are reported in the reviewed literature. In this section, the techniques used in each reference reviewed are summarized.

Table 2. Main ML-based approaches to the classification of requirements according to their Functional/Non-Functional nature.

RE Category	RE Activity	Main Approaches Used for NLP and Feature Extraction from NL Text, and for Dataset Preparation, and Main ML Approaches Used for Achieving the RE Activity
Classification of Requirements according to their Functional/ Non-Functional nature	Binary FR/NFR Classification, and ternary classification FR/NFR/Non-Req	TF; BoW; TF-IDF; Word2Vec; FastText; Doc2Vec; SMOTE; PCA; RST; DT; SVM; KNN; NB; MNB; NN; RF; K-means; Hierarchical Clustering; SVC; Bagged KNN; Bagged DT; Bagged NB; ExtraTree; GNB; SGD; GB; XGBoost; AdaBoost; ANN; RNN; LSTM; Bi-LSTM; MLP; CNN; BERT; PRCBERT; NoRBERT; MLM-BERT; GPT; BoW + MNB; TF-IDF + SVM; Doc2Vec + MLP + CNN; BoW + SVM; BoW + KNN; TF-IDF + SGD; Multiple correlation coefficient-based DT; Bi-LSTM-Att (Bi-LSTM + Attention Model); Ensemble Grid Search classifier using 5 models (RF, MNB, GB, XGBoost, AdaBoost); Trans_PRCBERT (PRCBERT fine-tuned on PROMISE); TPOT; Ensemble classif. combining 5 models (NB, SVM, DT, LR, SVC); Self-attention Bidirectional-RNN Deep Model (SABDM); Bidirectional Gated Recurrent Neural Networks (BiGRU); BERT-CNN.
	Classifying NFR in further subcategories, and Classifying ASFR	

The authors in [33] studied several ML approaches to distinguish between FR and NFR. Their approach included data cleansing, normalization and text preprocessing and vectorization steps, in which BoW, Term frequency-inverse document frequency (TF-IDF), featurization and ML Models, ROC and AUC curves, Bi-Grams and n-Grams in Python, Word2Vec and confusion matrix were used. According to the authors, the combination of BoW and MNB provided the best performance for binary classification.

The authors in [34] argue that existing techniques for classifying FR and NFR consider only one feature at a time, thus not being able to consider the correlation of two features, and so they are biased. In their study, they compare and extend ML algorithms to classify requirements, in terms of precision and accuracy, and have observed that the DT algorithm can identify different requirements and outperform existing ML algorithms. As the number of features increases, the accuracy using the DT is improved by 1.65%. To address the DT's limitations, they propose a multiple correlation coefficient-based DT algorithm. This approach, when compared to existing ML approaches, improves performance and accuracy.

In [35], the authors used a zero-shot learning (ZSL) approach to classify requirements into Functional and Non-Functional requirements, and to identify NFR categories, including security non-functional requirements, without using any labeled training data. The study shows that the ZSL approach achieves an F1-score of 0.66 for the FR/NFR classification task. For the NFR task, the approach yields an F1-score between 0.72 and 0.80, considering the most frequent classes.

In [36], TF-IDF and Word2Vec were the feature-extraction techniques used after the Natural Language (NL) text-preprocessing phase. The study then compared different ML algorithms to assess their precision and accuracy in classifying software requirements, namely DT, RF, LR, Neural Networks (NN), KNN and SVM. The results showed that the TF-IDF feature-selection algorithm performed better than the Word2Vec algorithm in subsequent classification algorithms.

The study in [37] implemented an ensemble technique using Grid Search classifier that can automatically tune the best parameters of the low-performing classifier. The goal was to use a fine-tuned ensemble technique combining five different models, namely RF, MNB, Gradient Boosting, XGBoost, and AdaBoost, to classify requirements into FR or NFR.

The study in [38] used an RNN-based model, namely Bidirectional Long Short-Term Memory (Bi-LSTM). This algorithm combines the forward and backward hidden layers to solve the sequential task better than LSTM. By combining Bi-LSTM and self-attention mechanisms, the authors noticed an improved requirements classification accuracy. The Bi-LSTM model was trained with the GloVe model. The architecture proposed in [38], named Self-attention-based Bidirectional-RNN Deep Model (SABDM), integrates NLP, Bi-LSTM and the self-attention mechanism, and was developed to improve the performance of Deep Learning in classifying requirements, both FR/NFR categorizations and within NFR categories.

Most studies deal with requirements classification as binary or multiclass classification problems and not as multilabel classification, which would allow a requirement to belong to multiple classes at the same time. As a way of minimizing preprocessing and to enable multilabel classification of requirements, in [41] a recurrent neural network-based Deep Learning system was used, namely Bidirectional Gated Recurrent Neural Networks (BiGRU). The authors investigated the usage of word sequences and character sequences as tokens. Using word sequences as tokens achieved results similar to the state of the art, effectively classifying requirements into functional and different non-functional categories with minimal text prepossessing and no feature engineering.

The authors in [43] present a Deep Learning pipeline to automatically classify functional requirements from an SRS. They highlight challenges posed by domain-specific terminology and employ Word2Vec, FastText and Doc2Vec embeddings, both pre-trained and retrained on internal data, to vectorize sentences. These embeddings feed into MLP and CNN classifiers, with the retrained CNN model achieving the highest accuracy at 77%.

In [44], the authors used Term Frequency, BoW and TF-IDF, together with four supervised and two unsupervised ML algorithms to classify requirements specifications into FR and NFR. When using BoW, the authors observed an accuracy of 0.725 with K-Nearest Neighbors (K-NN), 0.835 with Support vector machines (SVM), 0.849 with Logistic Regression (LR), 0.543 with K-means, 0.839 with Multinomial Naïve Bayes and 0.560 with Hierarchical clustering. Accuracy achieved with agglomerative clustering using TF-IDF was 0.797 with K-NN, 0.876 with SVM, 0.845 with LR, 0.470 with K-means and 0.856 with Multinomial Naïve Bayes. The authors conclude that, for better results, it is best to combine an SVM algorithm with TF-IDF. The authors also conclude that ML algorithms are suitable for classifying requirements on simple problems, but that for addressing larger problems it is necessary to apply rules-based AI models.

The research reported in [45] presents a Bidirectional Encoder–Decoder Transformer–Convolutional Neural Network (BERT-CNN) model for requirements classification. The convolutional layer is stacked over the BERT layer for performance enhancement. In order to extract features from requirement statements, the study employs CNN in task-specific layers of BERT. Experiments using the PROMISE dataset evaluated the solution's performance through multi-class classification of four key classes: operability, performance,

security and usability. Results showed that the BERT-CNN model outperformed the standard BERT approach when compared to existing baseline methods.

The studies in [46,53] use five distinct word-embedding techniques for classifying FR and NFR (quality) requirements. The Synthetic Minority Oversampling technique (SMOTE) is used to balance classes in the dataset used. Some dimensionality-reduction techniques are also used, namely Principal Component Analysis (PCA), which is used for reducing dimension, and Rank-Sum Test (RST), which is used for feature selection, to eliminate redundant and irrelevant features. Then, the vectors resulting from the word-embedding techniques used were provided as inputs to eight different classifiers for requirements categorization: Bagged K-Nearest Neighbors, Bagged Decision Tree, Bagged Naïve-Bayes, Random Forest, Extra Tree, Adaptive Boost, Gradient Boosting and a Majority Voting ensemble classifier, with DT, KNN and Gaussian Naïve Bayes (GNB). The authors conclude that a combination of word-embedding and feature-selection techniques with the various classifiers is successful in accurately classifying functional and quality software requirements [46,53].

In [47], the use of PRCBERT, or Prompt learning for Requirement Classification using BERT, is proposed. This approach applies flexible prompt templates to classify software requirements from small-sized requirements datasets (PROMISE and NFR-Review), and then adapts it to auto-label unseen requirements categories of their collected large-scale requirement dataset NFR-SO. Experiments conducted on PROMISE, NFR-Review and NFR-SO datasets enable one to conclude that PRCBERT exhibits moderately better classification performance than NoRBERT and MLM-BERT (BERT with the standard prompt template).

In [48], the authors propose applying ML and active learning (AL) to classify requirements in a given dataset, introducing the MARE process, which utilizes Naïve Bayes as the classifier. AL employs uncertainty sampling strategies to determine which data points should be labeled by the "oracle". Three AL strategies are explored: Least Confident (LC), Margin Sampling (MS) and Entropy Measure (EM). Experiments using two datasets were conducted to evaluate the performance of the MARE process. The findings suggest that better organization and documentation of requirements improve classification results. However, significant progress is still needed to develop a system capable of categorizing requirements with minimal human intervention at different levels of abstraction.

The work in [49] presents a proposal for the automated classification of quality requirements. The study involved the training and hyperparameter optimization of different ML models, with the user feedback classification. The study leverages the inherent knowledge of software requirements to train various ML algorithms using NLP techniques for information reuse, extraction and representation. The Tree-based Pipeline Optimization Tool (TPOT), an AutoML library developed by Olson et al. [156], which uses genetic algorithms, was employed to optimize ML models, improving fitness scores by up to 14%. TPOT achieved the highest weighted geometric mean (0.8363), followed by RF (0.82). However, applying these models to informal text requirements proved challenging, as automated classifiers struggled to achieve results above 0.3, highlighting the gap between machine and human classification performance.

The authors in [50] propose a technique to automatically classify software requirements using ML to represent text data from software requirements text and classify them as FR or NFR, based on BoW followed by SVM or KNN algorithms for classification. They experimented with the PROMISE_exp dataset, which includes labeled requirements, and observed that the use of BoW with SVM is better than using KNN algorithms with an average F-measure of all cases of 0.74.

The study in [51] looks for the automatic categorization of user feedback reviews into functional requirements and non-requirements. The study evaluates ML-based models to

identify and classify requirements from both formally written SRS documents and free text app reviews written by users. Similarly to other approaches, the work uses ML algorithms (SVM, SGD and RF) to identify and classify requirements, combined with NLP techniques, namely TF-IDF, to preprocess the requirements text.

In [52], an analysis of supervised ML models combined with NLP techniques is proposed to classify FRs and NFRs from large SRS. Experiments were conducted on the PROMISE dataset, in two phases: first, the focus was on distinguishing between FRs and NFRs; then, the aim was to classify NFRs into nine specific subcategories. The results show that SVM with TF-IDF achieved the best performance for FR classification, while SGD with TF-IDF was the most effective for NFR classification. For subclassifying NFRs, SVM with TF-IDF yielded the best results for Availability, Look and Feel, Maintainability, Operational and Scalability. Meanwhile, SGD with TF-IDF performed best for Security, Legal, and Usability, whereas RF with TF-IDF excelled in classifying Performance-related NFRs.

In [54], a new ensemble ML technique is proposed, combining different ML models and using enhanced accuracy as a weight in the weighted ensemble voting approach. The five combined models were NB, SVM, DT, LR and Support Vector Classification (SVC). When using the ML-based classifiers with the highest accuracies (SVM, SVC and LR), these yielded the same accuracy of 99.45% with the proposed ensemble, only the time improved when using a smaller number of classifiers.

In [55], the authors propose Requirements-Collector, a tool for automating the identification and classification of FRs from requirements specification and user feedback analysis. The Requirements-Collector approach involves ML and DL computational mechanisms. These components are intended to extract and preprocess text data from datasets of previous works, containing requirements, and then classify FR and NFR requirements. Preliminary results have shown that the proposed tool is able to classify RE specifications and user review feedback with reliable accuracy.

The work in [60] proposes an approach for classifying ASFRs, which are FR that contain comprehensive information to aid architectural decisions, and thus have a significant impact on the system's architecture. ASFRs are hard to detect, and if missed can result in expensive refactoring efforts in later stages of software development. The work presents experiments with a Deep Learning-based model for identifying and classifying ASFRs. The approach (Bi-LSTM-Att) applies a Bi-LSTM to capture the context information for each word in the software requirements text, followed by an attention model to aggregate useful information from these words in order to obtain the final classification. For ASFR identification, the Bi-LSTM-Att model yielded an F-score of 0.86, and for ASFR classification an F-score of 0.83, on average [60]. The authors also noted that Bi-LSTM-Att outperformed the baseline RAkEL NB classifier for all the labels, with industrial size datasets, although RAkEL NB seems to perform well on less data.

In [56], the authors propose an intelligent chatbot for keeping a conversation with stakeholders in NL and automating the requirements elicitation and classification yielding formal system requirements from the interaction. Afterwards, a classifier classifies the elicited requirements into FR and NFR. The collected requirements are written in unstructured free-flow English sentences, which are preprocessed to identify requirements, through the use of NLP and Dialogue Management, Rasa-NLU and Rasa-Core open source frameworks. After requirements elicitation by the chatbot, two classifiers were implemented, MNB and SVM, to categorize the elicited requirements into FR and NFR. The results show that MNB has better accuracy, precision, recall and F1-score than SVM (0.91 vs. 0.88 in all performance indicators) [56].

The authors in [61] study the application of ANN and a CNN to classify NFRs into five categories: maintainability, operability, performance, security and usability. They evaluated

their work on two widely used datasets with approximately 1000 NFRs. The results show that the implemented CNN model can classify NFR categories with a precision ranging between 82% and 94%, a recall indicator between 76% and 97%, and an F-score between 82% and 92%.

In [57], RF and gradient boosting algorithms are explored and compared to determine their accuracy in classifying functional and non-functional requirements. RF and gradient boosting are ensemble algorithms in ML. These combine the results from multiple base or weak learners to produce a final prediction, enabling one to improve accuracy and other indicators of prediction performance. Experimental results show that the gradient boosting algorithm improved prediction performance better than RF, when classifying NFR. However, the RF algorithm is more accurate in classifying FR.

In [58], the efficacy of ChatGPT in several aspects of software development is assessed. For requirements analysis, ChatGPT's proficiency in identifying ambiguities, distinguishing between FR and NFR, and generating use case specifications, was evaluated. The assessment, which was qualitative and subjected to the authors' opinion, revealed that ChatGPT has potential in assisting various activities throughout the SDLC, including requirements analysis, domain modeling, design modeling and implementation. The study also identified non-trivial limitations, such as a lack of traceability and inconsistencies among produced artifacts, which require human involvement. Overall, the results suggest that, when combined with human developers to mitigate the limitations, ChatGPT can serve as a valuable tool in software development [58].

5.2.2. Supporting Requirements Elicitation

Requirements elicitation is a requirements engineering phase, or set of activities, that deals with capturing, identifying and registering requirements. It helps to derive and extract information from stakeholders or other sources. It is an essential phase in building commercial software.

Table 3 presents the main NLP and ML-based approaches for supporting requirements elicitation. The table illustrates the techniques reported in the reviewed literature, which are summarized in this section.

Table 3. Main ML-based approaches to supporting requirements elicitation.

RE Category	RE Activity	Main Approaches Used for NLP and Feature Extraction from NL Text, and for Dataset Preparation, and Main ML Approaches Used for Achieving the RE Activity
Supporting Requirements Elicitation	Categorize and Classify Business Rules (as a source for RE)	TF; BoW; TF-IDF; Word2Vec; FastText; Doc2Vec; Rasa-NLU; Rasa-Core; MNB; SVM; Multi-dataset training; zero-shot approaches; Having a perturbator and a classifier positively influencing each other; Multimodal Autoencoder and Multimodal Variational Autoencoder methods; Supervised ML models; BERT; GPT; DL; NN; Transformer-based DL models; Neural Language Models; Hierarchical cluster labeling; Speech-acts based analysis technique; Part-of-Speech (POS) Tagging; NLP + ML techniques leveraging the concept of requirement boilerplate; NLP4ReF; NLP4ReF-NLTK; NLP4ReF-GPT; Trained LSTM RNN model (based on Rasa) + MNB or SVM; BERT-based transformer + static preference linker.
	Predicting/recommending Techniques for Req. Elicitation	
	Generating questions for Requirements elicitation	
	Identifying/Generating new Requirements from existing Requirements or from User Feedback; Identifying ASFR; Classifying User Feedback	

Business rules, which give body to the description of the business processes, can be an important source of software requirements specifications. The authors in [62] propose an approach to categorize and classify business rules based on Witt's approach, which classifies business rules into four main categories: definitional (or structural) rules, data rules, activity rules and party rules [62]. They conclude that the proposed approach

showed good accuracy, recall and F1-score values, when compared to the state-of-the-art approaches [62].

The requirements list resulting from the requirements elicitation phase is used as input for requirements analysis and management activities. Multiple elicitation techniques may be applied alternatively or in conjunction with other techniques to accomplish the elicitation. The prediction or recommendation of the best technique for requirements elicitation influences the requirements engineering approach. The authors in [63] analyze the current practices of requirements elicitation techniques application in practical software development projects and define factors influencing the technique selection based on the two-classification ML model and predict the usage of a particular elicitation technique depending on the project attributes and business analyst background. They conducted a survey involving 328 specialists from Ukrainian Information Technology (IT) companies. Gathered data was used to build and evaluate the prediction models.

According to the authors in [64], integrating advanced models like GPT-3.5 into RE remains largely unexplored. With the goal of exploring the capabilities and limitations of GPT-3.5 in software requirements engineering, the research presented in [64] investigates the effectiveness of GPT-3.5 in automating key tasks within RE. The authors identify the limitations of using GPT-3.5 in the requirement-gathering process and conclude that GPT-3.5 demonstrates proficiency in aspects like creative prototyping and question generation, but has limitations in areas like domain understanding and context awareness. The authors offer recommendations for future research focusing on the seamless integration of GPT-3.5 and similar models into the broader framework of software requirements engineering [64].

The intelligent conversational chatbot described in [56] and already mentioned before extracts system requirements, prompts for missing details and logs each requirement for automated requirements elicitation through natural language dialogue. After eliciting requirements, the chatbot, built on Rasa-NLU/Core and an LSTM-based RNN, then vectorizes requirements (BoW + TF-IDF) and classifies them into FR vs. NFR using MNB and SVM, with MNB outperforming SVM across accuracy, precision, recall and F1. The current system covers a limited domain and requires additional training data for broader applicability.

Addressing requirement defects early in RE is far more cost effective than fixing them later. In [39], the authors introduce Natural Language Processing for Requirements Forecasting (NLP4ReF), a suite of NLP/ML tools (NLP4ReF-NLTK and NLP4ReF-GPT) that reduce missing and incorrectly expressed requirements, minimizing requirements changes during the SDLC and ensuring that requirements accurately reflect stakeholder needs. NLP4ReF tools support initial requirements organization, FR/NFR classification, system-class identification and generation of overlooked requirements. Evaluations show these algorithms both generate many relevant new requirements and accurately classify existing ones, while the integration of Model-Based Systems Engineering (MBSE) diagrams offers a structured framework that deepens understanding and guides further tool development.

Besides addressing requirements classification between FR/NFR, as seen in Section 5.2.1, the authors in [51] also seek to automate the process of extracting functional requirements and filtering out non-requirements from user app reviews. Their proposal evaluates ML-based models to identify and classify software requirements from both formal Software SRS documents and mobile app user reviews. Initial evaluation of the ML-based models show that they can help classify user app reviews and software requirements as FR, NFR or non-requirements.

The research in [55], already mentioned before when addressing requirements classification, also intends to automatically identify, extract and preprocess text containing requirements and user feedback, to generate requirements specification. The proposed Requirements-Collector tool uses ML- and DL-based approaches to automatically classify

requirements discussed in RE meetings (stored in the form of audio recordings) and textual feedback in the form of user reviews. The authors argue that the Requirements-Collector tool has the potential to renovate the role of software analysts, which can experience a substantial reduction of manual tasks, more efficient communication, dedication to more analytical tasks and assurance of software quality from conception phases [55].

Developers frequently elicit requirements from user feedback, such as bug reports and feature requests, to help guide the maintenance and evolution of their products [65]. By linking feedback to their existing documentation, development teams enhance their understanding of known issues and direct their users to known solutions. The authors in [65] apply Deep Learning techniques to automatically match forum posts with related issue tracker entries, using an innovative clustering technique. Strong links between product forums, issue trackers and product documentation have been observed, forming a requirements ecosystem that can be enhanced with state-of-the-art techniques to support users and help developers elicit and document the most critical requirements [65].

Studies to elicit stakeholder preferences were developed in [66], using scenarios where users describe their goals for using directory services to find entities of interest, such as apartments, hiking trails, etc. The article's results reveal that feature support for preferences varies widely among directory services, with around 50% of identified preferences unmet. The study also explored automatic preference extraction from scenarios using named entity recognition across three approaches, with a BERT-based transformer achieving the best results (81.1% precision, 84.4% recall and 82.6% F1-score on unseen domains). Additionally, a static preference linker was introduced, linking extracted entities into preference phrases with 90.1% accuracy. This pipeline enables developers to use the BERT model and linker to identify stakeholder preferences, which can then inform improvements and new features to better address gaps in service.

In [67], ML classifiers are used to classify bug reports and feature requests using seven datasets from previous studies. The authors evaluate classifiers' performance on users' feedback from unseen apps and entirely different datasets and they assess the impact of channel-specific metadata. They find that using metadata as features in classifying bug reports and feature requests rarely improves performance and while classification is similar for seen and unseen apps, classifiers struggle with unseen datasets. Multi-dataset training or zero-shot approaches can somewhat alleviate this issue, with implications on user feedback classification models for extracting software requirements.

In [68], the authors propose an approach to creatively generate requirements candidates via the adversarial examples resulting from applying perturbations to the original requirements descriptions. In the presented architecture, the perturbator and the classifier positively influence each other. Each adversarial example is uniquely traceable to an existing feature of the software, instrumenting explainability. The experimental evaluation used six datasets and shows that around 20% adversarial shift rate is achievable [68].

Several works investigate which techniques and ML models are most appropriate for detecting relevant user feedback and reviews and for classifying, embedding, clustering and characterizing those reviews for generating requirements across multiple feedback platforms and data domains [69–74,76].

The study in [69] explores unimodal and multimodal representations across various labeling levels, domains and languages to detect relevant app reviews using limited labeled data. It introduces a one-class multimodal learning method requiring labeling only relevant reviews, thus reducing the labeling effort. To enhance feature extraction and review representation with fewer labels, the authors propose the Multimodal Autoencoder and the Multimodal Variational Autoencoder methods, which learn representations that combine textual and visual information based on reviews' density. Density information can be

interpreted as a summary of the main topics or clusters extracted from the reviews [69]. The studied methods achieved competitive results using just 25% of labeled reviews compared to models trained on complete datasets, with multimodal approaches reaching the highest F1-score and AUC-ROC in twenty-three out of twenty-four scenarios.

In [70], the authors investigate whether enterprise software vendors can elicit requirements from their sponsored developer communities through data-driven techniques. The authors collected data from the SAP community and developed a supervised ML classifier for automatically detecting feature requests of third-party developers. Based on a manually labeled dataset of 1500 questions, the proposed classifier reached a high accuracy of 0.819. Their findings reveal that supervised ML models may be an effective means for the identification of feature requests.

In [71], the authors propose using the state-of-the-art transformer-based DL models to automatically classify sentences in a discussion thread. The authors propose a benchmark to ensure standardized inputs for training and testing for this problem. They conclude that their Transformer-based classification proposal significantly outperforms the state of the art [71].

The approach presented in [73] proposes a hierarchical cluster-labeling method for software requirements that leverages contextual word embeddings. This method addresses previous issues such as duplicate requirements from user reviews and the challenges of handling different granularity levels that obscure hierarchical relationships between software requirements. The authors use neural language models to create semantically rich representations of software requirements, clustering them into groups and subgroups based on similarity in the embedding space. Representative requirements are then selected to label each cluster and sub-cluster, effectively managing duplicate entries and different granularity levels [73].

Within the RE process of defining, documenting and maintaining software requirements, the authors in [74] focus on the problem of automatic classification of CrowdRE into sectors. CrowdRE involves large-scale user participation in RE tasks. The authors' proposal involves three different approaches for sector classification of CrowdRE, based on supervised ML models, NN and BERT, respectively. Classification approaches were applied to a CrowdRE dataset, comprising around 3000 crowd-generated requirements for smart home applications. The obtained performance is similar to several other classification algorithms, indicating that the proposed algorithms can be very useful for categorizing crowd-based requirements into sectors [74].

Although initial progress has been made in using mining techniques for requirements elicitation, it remains unclear how to extract requirements for new apps based on similar existing solutions and how practitioners would specifically benefit from such an approach.

In [76], the authors focus on exploring information provided by the crowd about existing solutions to identify key features of applications in a particular domain. The discovered features and other related influential aspects (e.g., ratings) can help practitioners to identify potential key features for new applications [76]. The authors present an early conceptual solution to discuss the feasibility of their approach.

Online user reviews and feedback (tweets, forum posts) are first filtered with NLP to remove noise and only then are extracted and classified (e.g., bug report vs. feature request). In [77], the authors refine a speech act-based analysis technique and evaluate it on two datasets taken from an open source software project (161,120 textual comments) and from an industrial project in the home energy-management domain. Their approach classifies messages into "Feature/Enhancement" and "Other" with F-scores of 0.81 and 0.84, respectively, and demonstrates clear links between specific speech acts, issue categories and priority levels.

To advance software creativity, several techniques have been proposed, such as multi-day workshops with experienced requirements analysts and semi-automated tools that support focused creative thinking. The authors in [75,78] propose a novel framework for providing an end-to-end automation to support creativity in both new and existing systems. The framework reuses requirements from similar software freely available online, uses advanced NLP and ML techniques and leverages the concept of requirement boilerplate to generate candidate creative requirements. The framework has been applied on three application domains, Antivirus, Web Browser and File Sharing, and further reports a human subject evaluation. The results exhibit the framework's ability to generate creative features even for a relatively mature application domain, such as Web Browser and provoke creative thinking among developers irrespective of their experience levels.

Software companies need to quickly fix reported bugs and release requested new features or they risk negative reviews and reduced market share. The sheer volume of online user feedback renders manual analysis impractical. The authors in [79] note that online product forums are a rich source of user feedback that may be used to elicit product requirements. The information contained in these forums often includes detailed context for specific problems that users encounter with a software product. By analyzing two large forums, the study in [79] identifies 18 distinct types of information (classifications) relevant to maintenance and evolution tasks. The authors found that a state-of-the-art App Store tool cannot accurately classify forum data, underlining the need for specialized techniques to extract requirements from product forums. In an exploratory study, they developed classifiers incorporating forum-specific features, achieving promising results across all classifiers, with F-scores ranging from 70.3% to 89.8%.

5.2.3. Improving Quality of Requirements and Software

Most software requirements are written using natural language, which has no formal semantics and has a high risk of being misunderstood due to its natural tendency towards ambiguity and vagueness. Improving the quality of requirements and software involves reducing the ambiguity, incompleteness, non-uniqueness and coverage (in terms of users' needs) of the requirements specification. It also involves identifying requirements that may have effects regarding sustainability, security and usability of the future system, besides other quality characteristics. Another way of improving the requirements quality is registering and monitoring their inter-dependencies and pre- and post-conditions.

The quality of the software and the adherence of planned or developed software features to the stated requirements may also be addressed through validation tests. These may be, at least partially, drawn from the SRS. And the probability of defects can also be predicted.

Table 4 presents the main approaches for the NLP phase, along with the main ML-based approaches for improving the quality of requirements and software. The table shows the main techniques reported in the reviewed literature. Each of the references reviewed is summarized in this section.

The work in [58], mentioned above, also targets the quality improvement of requirements and software. The efficacy of ChatGPT regarding identifying ambiguities and generating accurate use case specifications, and fixing errors encountered during the software implementation, was assessed. As mentioned before, the study also identified a lack of traceability and inconsistencies among produced artifacts, which do not dispense with human involvement.

Table 4. Main ML-based approaches to improving the quality of requirements and software.

RE Category	RE Activity	Main Approaches Used for NLP and Feature Extraction from NL Text, and for Dataset Preparation, and Main ML Approaches Used for Achieving the RE Activity
Improving Quality of Requirements and Software	Requirements itemization, simplification, disambiguation; Detecting incompleteness, ambiguity, inaccuracy, implicit requirements, semantic similarity, and other risks (smells) in Req. Spec.; Coping with ambiguity through the use of controlled vocabulary and Ontologies	Tokenization; POS Tagging; Dependency Parsing; Data balancing techniques such as SMOTE, RUS, ROS, and Back translation (BT); LLM; GPT; ACM; SVM; LogR; MNB; FPDM using Adaptive Boosting, Gradient Boosting, and Extreme Gradient Boosting; BASAALT / FORM-L; NLP4ReF-NLTK and NLP4ReF-GPT; NLP techniques with features extracted using TF-IDF and BoW + Various classifiers (LR, NB, SVM, DT, KNN); BERT's masked language model to generate contextualized predictions + ML-based filter to post-process BERT's predictions; Text classification technique; Sentence embedding and antonym-based approach for finding incomplete Reqs.; BERT-based and clustering approach for detecting intra- or cross-domain ambiguities; ULMFiT (Transfer learning approach where the model is pre-trained to a general-domain corpus and then fine-tuned to classify ambiguous vs unambiguous reqs); COTIR (integrates Commonsense knowledge, Ontology and Text mining for early detecting Implicit Reqs.); ML approach to extract UX characteristics from FR; BERT + knowledge graphs integrating information from various sources on security and vulnerabilities + Transfer learning is applied to reduce the training demands of ML and DL models; Adaboost ensemble method; ML algorithms to predict vulnerabilities for new reqs; ML-based defect prediction models using design-level metrics and data sampling techniques; Knowledge engineering-based architecture to create a traceability matrix using NLP and ML techniques, using an ontology and optimization algorithm, including real world knowledge and not requiring a lot of data; Test case generation using text classification with NB algorithm, Scikit-learn and NLTK to identify preconditions and postconditions within requirements; Combining NLP and ML to automate software requirement-to-test mapping; Supervised ML classifiers to classify user stories according to valuable and testable metrics; TF-IDF + LR achieved highest performance for req. smells classif.; TF-IDF + SVM outperformed other algorithms; ULMFiT achieved higher accuracy than SVM; LogR; MNB classifiers; COTIR outperforms other IMR tools.
	Verification of Quality Characteristics in NFR (e.g., usability, UX, security, explainability) and user feedback; Ensuring security of Requirements; GDPR; Assessing the SRS quality by ISO 25010; Test case generation / Automate the quality checking/analysis of a Req./user story	
	Identify potential effects on Sustainability; Assess Transparency and Sustainability as NFR	
	Verification of pre-/post-conditions of Requirements; Requirements to Test Cases Traceability; Predicting probability of defects using design-level attributes	
	Classification of requirements in two classes: "Integration Test" and "Software Test" using Machine Learning approaches	

One recurrent difficulty in requirements analysis is requirements itemization or simplifying/subdividing requirements. This difficulty arises due to the inherent ambiguity and redundancy of requirements described in natural language. It is very important to determine the list of itemized requirements from the requirements document. The method in [80] addresses the challenge of itemizing natural language requirements and extracting discrete requirement entries. By combining NLP techniques with ML models to mimic an expert's three-step process (locating requirement boundaries, building extraction models and deriving fine-grained semantics), the approach achieved nearly 80% accuracy on military domain texts, promising faster, easier requirement itemization for practitioners.

In [81], an ML-based approach to formalizing requirements written in natural language text is proposed. The approach targets critical embedded systems and extracts information to assess the quality and complexity of requirements. The authors used the open source NLP framework spaCy for tokenization, Part-of-Speech (PoS) tagging and dependency parsing based on a pre-trained English language model. Then, a phase for identifying text chunks follows, which uses a rule-based exploration approach in contrast to domain-specific training alternatives. According to the authors, this is to ensure independence of a specific engineering domain. Text chunks are then put into a normalized order. If this is not possible, a quality issue may be detected. The normalized sequence of chunks can be used for "building test oracles, for the implementation of software and for applying metrics so that requirements may be compared for similarity or be evaluated" [81].

In [82], the authors propose a model to detect fault-prone software requirements specifications, consisting of two main components: (1) an Ambiguity Classification Module (ACM); and, (2) a Fault-Prone Detection Module (FPDM). The ACM selects the best Deep Learning algorithm to classify requirements as ambiguous or clean, identifying various types of ambiguity (lexical, syntactic, semantic and pragmatic). Then, the FPDM uses key SRS components, such as title clarity, description, intended users and the ambiguity classification, to detect fault-prone requirements. The ACM achieved an accuracy of 0.9907 and the FPDM 0.9750. To further enhance detection, particularly for edge/cloud applica-

tions, the authors applied boosting algorithms (Adaptive Boosting, Gradient Boosting and Extreme Gradient Boosting), improving accuracy by leveraging SRS features. They also propose a fault-prone severity scale that categorizes ambiguity as low, moderate or high based on a calculated score from key SRS elements [82].

The use of NLP4ReF, proposed in [39], also targets the disambiguation of requirements specifications. It uses ML and NLP to identify duplicate, incomplete and hidden requirements. NLP4ReF-NLTK and NLP4ReF-GPT algorithms are used to classify requirements and generate new relevant requirements.

The study in [83] underscores that fully automating requirement disambiguation is not feasible. Human review remains essential. Using the BASAALT method and FORM-L language, the authors formalize requirements into precise, simulable models that integrate missing context. Behavioral simulations with tools like Stimulus then reveal defects such as inadequate, overly ambitious or contradictory requirements. Stakeholders review these simulation results to validate alignment with intended needs. While effective at detecting semantic and formal issues, the process currently depends on manual application of BASAALT/FORM-L for identifying ambiguities and crafting corrected requirements.

The study in [84] addresses "requirement smells" (indicators of ambiguity or vagueness) by training ML models both to detect and to prioritize them. Using a corpus of 3100 expert-labeled requirements, the authors extract TF-IDF and BoW features and evaluate classifiers (LR, NB, SVM, DT, KNN). They identify ten smell categories and rank them by severity and requirement importance. LR with TF-IDF achieved the highest performance with 94% accuracy for requirement smells classification. For requirement smells prioritization, SVM outperformed other algorithms with 99% accuracy.

Detecting incompleteness in NL requirements is a major challenge. In [85], the authors investigate using BERT's masked language modeling to detect missing information in natural language requirements by withholding key content and evaluating BERT's ability to predict the omitted terms. They determine the optimal number of predictions per mask and introduce an ML-based filter to reduce noise in BERT's suggestions. Evaluated on 40 specifications from the PURE dataset, their approach outperforms simple baselines at highlighting omissions and the filter further improves completeness checking.

The problem with natural language is that it can easily lead to different understandings if it is not expressed precisely by the stakeholders involved [86]. This may result in building a product that is not aligned with the stakeholders' expectations.

The work in [86] tries to improve the quality of the software requirements by detecting language errors based on International Standards Organizations (ISO) 29,148 requirements language criteria. The proposed solution is based on previous existing solutions, which apply classical NLP approaches to detect requirements' language errors. In [86], the authors seek to improve the previous work by creating a manually labeled dataset and using ensemble learning, DL and other techniques, such as word embeddings and transfer learning to overcome the generalization problem that is tied with classical NLP and improve precision and recall metrics using a manually labeled dataset.

The work in [87] also addresses duality and incompleteness in NL software requirements specification. Different from previous approaches, in [87] the authors focus on the requirements incompleteness implied by the conditional statements and propose a sentence-embedding and antonym-based approach for detecting the requirements incompleteness. The guiding idea is that when one condition is stated, its opposite condition should also be there or else the requirements specification is incomplete. Hence, the proposed approach starts by extracting the conditional sentences from the requirements specification and eliciting the conditional statements which contain one or more conditional expressions. Then, conditional statements are clustered using the sentence-embedding technique and the con-

ditional statements in each cluster are further analyzed to detect potential incompleteness, by using negative particles and antonyms [87]. The results of the proposed approach have shown a recall of 68.75% and an F1-measure of 52.38%.

Ambiguity in requirement engineering document may lead to disastrous results, thereby hampering the entire development process and ending up compromising the quality of a system. In [88], the authors discuss the types of ambiguity found in the RE document and approaches to handling and providing a level of automatic assistance in reducing ambiguity and improving requirements. The study also confirms the use of a text-classification technique to classify a text as "ambiguous" or "unambiguous" at the syntax level. The objectives of the work were mainly to identify the presence of ambiguity in any RE document with the help of ML techniques and finally to minimize or reduce it.

The application of neural word embeddings for detecting cross-domain ambiguities in software requirements has recently gained significant attention. Several methods in the literature estimate how meaning varies for common terms across domains, but they struggle to detect terms used in different contexts within the same domain, i.e., intra-domain ambiguities or those in a requirements document of an interdisciplinary project. The work in [89] introduces a BERT-based and clustering approach to identify such ambiguities. For each context in which a term appears, the approach provides a list of similar words and example sentences illustrating its context-specific meaning. Applied to both a computer science corpus and a multi-domain dataset covering eight application areas, the approach has proven highly effective in detecting intra-domain ambiguities [89].

In [90], the authors used transfer learning by using ULMFiT, where the model was pre-trained to a general domain corpus and then fine-tuned to classify ambiguous vs. unambiguous requirements (target task). Back translation (BT) was also used as a text-augmentation technique to see if it improved the classification accuracy. The proposed model was then compared with ML classifiers like SVM, Logistic Regression (LogR) and MNB, and the results showed that ULMFiT achieved higher accuracy than those classifiers, improving the initial performance by 5.371%. The authors conclude that the proposed approach provides promising insights on how transfer learning and text augmentation can be applied to small datasets in requirements engineering.

The work in [91] addresses identification of implicit requirements (IMRs) in SRS. Implicit requirements are not specified by users, but may be crucial to the success of a software project. A software tool was developed, called COTIR, which integrates commonsense knowledge, ontology and text mining for early identification of implicit requirements. In [91] the authors demonstrate the tool and conclude that it relieves human software engineers from the tedious task of manually identifying IMRs in huge SRS documents. The performed evaluation shows that COTIR outperforms existing IMR tools [91].

Semantic similarity information supports requirements tracing and helps to reveal important requirements quality defects such as redundancies and inconsistencies [92]. The authors in [92] created a large dataset for analyzing the similarity of requirements, through the use of Amazon Mechanical Turk, a crowd-sourcing marketplace for micro-tasks. Based on this dataset, they investigate and compare different types of algorithms for estimating semantic similarities of requirements, covering both relatively simple bag-of-words and ML models. After experiments on their dataset, they conclude that the best performances were obtained by a model which relies on averaging trained word and character embeddings as well as an approach based on character sequence occurrences and overlaps.

In [93], the authors present techniques of NLP which work out greatly with resepct to extracting information properly and minimizing the bugs that may be generated in later parts of software development. Using techniques of NL Interpretation, software engineers

can outline the most accurate requirements of customers, which can improve the quality of requirements and ultimately of the resulting software product.

Several studies have addressed early certification of quality attributes from requirements text, such as usability, User Experience (UX) or security. In [94], an ML pipeline extracts UX-related features from initial requirements and uses RF to predict overall UX Key Performance Indicator (KPI)s, achieving an F1-score of 0.91. These KPI estimates closely match those from UX-oriented prototype-based evaluations, allowing one to conclude that the proposed model can evaluate UX instantly without interventions from end-users or UX designers [94]. Meanwhile, [99] presents a crowd-sourced UX benchmark dataset, validated for internal consistency (high Cronbach's Alpha) and accuracy (low root mean square error), to train models that estimate UX directly from requirements without user testing.

The work in [95] focuses on ensuring system security is addressed in the requirements management phase rather than leaving it for later phases in the software-development process. The authors propose an approach to combine useful knowledge sources like customer conversation, industry best practices and knowledge hidden within the software-development processes. BERT is used in the proposed architecture to utilize its language-understanding capabilities. The work also investigates the use of knowledge graphs to integrate information from various industry sources on security practices and vulnerabilities, ensuring that the requirements-management team stays informed with critical data. Additionally, transfer learning is applied to reduce the expensive training demands of ML and DL models. The proposed architecture was validated within the financial domain and agile development models. The authors propose that this approach could effectively integrate software requirements management with data science practices by leveraging the extensive information available in the software-development ecosystem.

Software security is also a major concern in the work presented in [96]. Based on the principle that the root of a system security vulnerability can often be traced back to the requirements specification, the authors advocate a novel framework to provide an additional measure of predicting vulnerabilities at earlier stages of the SDLC. In the study in [96], the authors build upon their proposed framework and leverage state-of-the-art ML algorithms to predict vulnerabilities for new requirements, together with a case study on a large open source software (OSS) system, Firefox, evaluating the effectiveness of the extended prediction module. The results show that the framework could be a viable complement to the traditional vulnerability-fighting approaches.

Complying with the EU GDPR can be a challenge for small- and medium-sized enterprises. The work reported in [97] considers GDPR compliance as a high-level goal in software development that should be addressed at the beginning of software development, that is, during RE. The authors argue that NLP can be used to automate this process and present initial work, preliminary results and the current state of art on verifying requirements' GDPR compliance.

In [98], the authors present a user feedback classifier based on ML for the classification of user reviews according to software quality characteristics compliant with the ISO 25010 standard. The proposed approach was achieved by testing several ML algorithms, features and class-balancing techniques for classifying user feedback on a dataset of 1500 reviews. The maximum F1- and F2-scores obtained were 60% and 73%, with recall as high as 94%. The authors conclude that the proposed approach does not replace human specialists, but helps in reducing the effort required for requirements elicitation.

The study in [100] reports on the development of a method of activity of ontology-based intelligent agent for evaluating initial stages of the software lifecycle. Based on the developed method, the intelligent agent evaluates whether the information in the SRS is sufficient or not. It provides a numerical assessment of the sufficiency level for

each non-functional feature individually and for all features overall, along with a list of attributes (measures) or indicators that should be added to improve the SRS's completeness or level of sufficiency [100]. In experiments, the agent analyzed the SRS for a transport logistics decision support system and determined that the information was not sufficient for assessing the quality by ISO 25010 and for assessing quality by metric analysis.

Software developers are gradually becoming aware that their systems have effects on sustainability [101]. Researchers are currently exploring approaches which strongly make use of expert knowledge to identify potential effects. In the work in progress research reported in [101], the authors looked at the problem from a different angle: they worked on the exploration of an ML-based approach to identify potential effects. Such an approach allows one to save time and costs but increases the risk that potential effects are overseen. First results of applying the ML-based approach in the domain of home automation systems are promising, but also indicate that further research is needed before the proposed approach can be applied in practice.

The growing complexity and ubiquity of software systems increase user reliance on their correct functioning, which in turn demands that these systems and their decision processes become more transparent. To achieve this, transparency requirements must be clearly understood, elicited and refined into lower-level requirements [102]. However, there is still limited understanding of how the requirements engineering process should address transparency, including the roles and interactions among UX designers, data scientists and other stakeholders. To address this gap, the work in [102] investigates the requirements engineering process for transparency through empirical studies with practitioners and other stakeholders. Preliminary findings indicate that further research is needed to develop effective solutions that support transparency in requirements engineering.

It is very important to deliver a defect-free product that matches all the requirements specified by the client and the product must pass all the test cases [103]. To do this systematically, a requirement traceability matrix is used. A requirement traceability matrix relates two items' lists in two dimensions. In this case, it shows the relations of all the requirements and the test cases, thus allowing one to track the relation between the customer's requirements for the system and the test cases for validating the requirements. There are many approaches to creating an efficient traceability matrix using language processing and ML techniques, but these approaches require a lot of data and do not take into account real-world knowledge and may lead to errors. In [103], the authors propose a knowledge engineering-based architecture to this problem with the use of ontology, ML and an optimization algorithm which produces a dependability and steadiness of 97% and 95%, respectively, and the performance of the proposed model is compared with baseline approaches.

The authors in [104] propose an approach for test case generation using text classification, using the NB algorithm to identify preconditions and postconditions within software requirements. The approach categorizes software requirements into two categories, "none" and "both", which indicate the presence or absence of preconditions and postconditions in software requirements. The research employs the NB algorithm, a widely used probabilistic classification algorithm in text-classification tasks [104]. It uses two libraries, namely Scikit-learn and Natural Language Toolkit (NLTK). The best accuracy score, which was obtained with the Scikit-learn model, was 0.86, which demonstrates the feasibility of reducing the effort and time required for classifying test case components based on software requirements. The proposed approach not only streamlines the identification of essential components in software requirements but also opens up possibilities for further automation and optimization of the testing process [104].

Accurate mapping of software requirements to tests is critical for ensuring high software reliability [105]. The dynamic nature of software requirements demands that these are traceable and measurable throughout the SDLC in order to be able to plan software tests and integration tasks, during the development phase, and the evaluation and verification tasks or the application of patches, during the operation phase. To address these challenges, a novel method is proposed in [105], combining NLP and ML to automate software requirement-to-test mapping. The proposed method formalizes the process of reviewing the recommendations generated by the automated system, enabling engineers to improve software reliability and reduce cost and development time [105].

In Agile software project-management methodologies, user requirements are frequently stated in the form of user stories, the smallest semi-structured specification units of user requirements. In [106], the authors automatically assess these stories against the "Testable" and "Valuable" INVEST checklist (https://scrum-master.org/en/creating-the-perfect-user-story-with-invest-criteria/ (accessed on 2 September 2024)) using supervised ML classifiers trained on industrial data. They apply balancing techniques (SMOTE, RUS, ROS, back translation) and find that, while accuracy and precision remain similar, recall improves markedly across all classifiers. This demonstrates ML's potential to help teams validate user stories and enhance software quality.

Providing automatic requirement analysis techniques for modeling and analyzing requirements is a must for saving manpower. In [107], a cloud service method for automated detection of quality requirements in SRS is proposed. The study also presents a novel approach for processing automatic classification of software quality requirements based on supervised ML techniques applied for the classification of training document and predict target document software quality requirements.

Approaches aiming to minimize vulnerabilities in software have been dominated by static and dynamic code-analysis techniques, often using ML. These techniques are designed to detect vulnerabilities in the post-implementation stage of the SDLC [96]. Accommodating changes after detecting a vulnerability in the system in later stages of the SDLC is very costly, sometimes even infeasible as it may involve changes in design or architecture. In [96], a framework to provide additional measures of predicting vulnerabilities at earlier stages of the SDLC is proposed.

In [108], a method for automatically generating test cases, for system testing and acceptance testing, from requirements is studied. The authors propose training the data-selection quality-improvement technique in the cosine similarity with the test data, and confirmed the effectiveness of the methods. A second method was also proposed that adds the application judgment technique by the standard deviation value. The proposed methods obtained the maximum value of accuracy with less training data.

Software used in communication systems is increasingly becoming more complex and larger in scale to accommodate various service requirements. Since telecom carrier networks serve as basic social infrastructures, it is important to maintain their reliability and safety as a critical lifeline [109]. The authors of [109] address the high cost and time of manual test case creation in large-scale telecom software by automating test case generation. Using existing test cases authored by expert engineers as training data, they train ML models to extract homogeneous test cases directly from requirements specifications, eliminating dependence on individual expertise. To compensate for limited training data, they enhance learning efficiency by carefully preparing and augmenting input data rather than expanding the dataset, demonstrating that this preprocessing method significantly improves the accuracy of automatically generated test cases.

Model analytics for defect prediction allows quality assurance groups to build prediction models earlier and to predict the defect-prone components before the testing phase for

in-depth testing [110]. In [110], it is shown that ML-based defect-prediction models using design-level metrics in conjunction with data-sampling techniques are effective in finding software defects. The study shows that design-level attributes have a strong correlation with the probability of defects and the SMOTE data-sampling approach improves the performance of prediction models. When design-level metrics are applied, the Adaboost Ensemble Method provides the best performance regarding detecting the minority class samples [110].

For quality assurance and maturity support of the final products, requirements must be verified and validated at different testing levels. To achieve this, requirements are manually labeled to indicate the corresponding testing level. The number of requirements can vary from a few hundred in smaller projects to several thousands in larger projects. Their manual labeling is time consuming and error-prone, thus sometimes incurring an unacceptable high cost. In [111], the initial results on an automated requirements classification approach proposal are reported. Requirements are automatically classified into two classes, using ML approaches: 'Integration Test' and 'Software Test'. The proposed solution may help the requirements engineers by speeding up the requirements classification and thus reducing the time to market of final products.

5.2.4. Extracting Knowledge from Requirements

The extraction of knowledge from requirements specifications enables obtaining semantically rich objects and concepts for building more formal models of such requirements or of the intended system. Several reviewed references target this goal, either for formalizing requirements, generating feature models, domain models or use case models, or to build domain vocabularies or ontologies that may help in further tasks towards designing and building the intended system.

In Table 5, the main NLP and ML-based approaches for extracting knowledge from requirements are presented. As seen in the table, several techniques are reported in the studied literature and these are addressed in this section, along with each studied reference.

Table 5. Main ML-based approaches to extracting knowledge from requirements.

RE Category	RE Activity	Main Approaches Used for NLP and Feature Extraction from NL Text, and for Dataset Preparation, and Main ML Approaches Used for Achieving the RE Activity
Extracting Knowledge from Requirements	Requirements Formalization; Extracting/Associating Features (Feature Extraction) or Model Elements from/to Requirements; Extract domain vocabulary from requirements for Feature Modeling or ontology construction	NLP techniques and ML algorithms; IE; GPT; BASAALT/FORM-L; TextRank (NLP techniques and ML algorithms); Ontology learning method (the ontology is semi-automatically constructed) + Information Entropy and CCM method; NB; Linear SVM; KNN; RF; MTBE techniques; LLM; READ; MNB; GNB; SVM; NER; NLP + WSL+ (RF or SVM or NB); Combining matching with automated reqs analysis and ROF; AGER System; Extension of the Siemens toolchain for ALM that creates trace links between requirements and models; DL-based, non-exclusive classification approach for FRs, using Word2Vec and FastText, and a CNN; SyAcUcNER; SRXCRM; ReqVec; BiLSTM NN to find relationships and patterns among sentences around domain concepts; DoMoBOT; DF4RT; TLR-ELtoR; OpenReq-DD dependency detection tool (NLP + ML); Cascade DF model integrating infor. retrieval (IR), query quality (QQ) and distance metrics; ML + Logical reasoning; Combination of evolutionary computation and ML techniques to recover traceability links between requirements and models; S2Trace (Unsupervised reqs traceability approach).
	Detecting or Extracting Requirements Dependencies / Requirements Traceability (forward and backward)	

Natural Language Processing (NLP) techniques have demonstrated their effectiveness in analyzing technical specification documents. One such technique, Information Extraction (IE), enables the automated processing of SRS by transforming unstructured or semi-structured text into structured data. This allows requirements to be converted into formal logic or model elements, enhancing their clarity and usability.

In [114], the authors introduce an IE method specifically designed for SRS data, analyze the proposed technique on a set of real requirements and exemplify how information obtained using their technique can be converted into a formal logic representation.

ChatGPT has also shown potential in assisting software engineers in extracting model elements from requirements specification. In [58], ChatGPT is used, among other things, for requirements analysis and domain and design modeling. The study identifies a lack of traceability and inconsistencies among produced artifacts as the main drawbacks. This demands human involvement in RE activities, but can serve as a valuable tool in the SDLC, enhancing productivity.

User demand is the key to software development. The domain ontology established by AI can be used to describe the relationship between concepts in a specific domain, which can enable users to agree on conceptual understanding with developers [112]. The study in [112] uses the ontology-learning method to extract the concept and the ontology is constructed semi-automatically or automatically. Because the traditional weight-calculation method ignores the distribution of feature items, the authors introduce the concept of information entropy and the CCM method is further integrated [112]. This method allows improving the automation degree of ontology construction and also makes the user requirements of software more accurate and complete.

The BASAALT/FORM-L approach, seen before, can also be used to extract knowledge from textual requirements [83]. The approach creates semantically precise and simulable models while integrating missing contextual details. These models can then be reviewed by stakeholders to ensure alignment with intended requirements.

In [81], a Machine Learning-based approach is proposed to formalize NL requirements for critical embedded systems. Using spaCy's pre-trained NLP model, the method performs tokenization, part-of-speech (PoS) tagging and dependency parsing. A rule-based strategy extracts text chunks, ensuring domain independence. These chunks are reordered into a standardized format, with deviations signaling quality issues. The structured output supports test oracle generation, software implementation and requirement assessment through similarity and complexity metrics.

In [113], the authors address how the production of model transformations (MTs) can be accelerated by automating transformation synthesis from requirements, examples and metamodels. A synthesis process is introduced, based on metamodel matching, correspondence patterns between metamodels and completeness and consistency analysis of matches [113]. The authors also address how to deal with the limitations of metamodel matching by combining matching with automated requirements analysis and model transformation by example (MTBE) techniques [113]. In practical examples, a large percentage of required transformation functionality can usually be constructed automatically, thus potentially reducing development effort. The efficiency of synthesized transformations is assessed [113].

The authors in [115] extend prior work with a Rule-Based Ontology Framework (ROF) that captures elicited requirements in a formal Requirements Ontology (RO) and then automatically generates both Business Process Model and Notation (BPMN) process models and IEEE-standard SRS documents. A case study on a university lecturer workload system demonstrates ROF's practicality, showing it significantly reduces the effort needed to produce complete specifications.

Given the rise of Large Language Model (LLM)s, the authors in [116] investigate their potential for extracting domain models from agile product backlogs. They compare LLMs against (i) a state-of-practice tool and (ii) a specialized NLP approach, using a dataset of 22 products and 1679 user stories. This research marks an initial step toward leveraging LLMs or tailored NLP for automated model extraction from requirements text [116].

Model-Based Software Engineering (MBSE) offers various modeling formalisms, including domain models, which capture key concepts and relationships in class diagrams, in early design stages. Domain modeling transforms informal NL requirements into concise,

analyzable models. However, existing automated approaches face three key challenges: insufficient accuracy for direct use, limited modeler interaction and a lack of transparency in modeling decisions [117]. To address this, the authors in [117] propose an algorithm that enhances bot–modeler interactions by identifying and suggesting alternative configurations. Upon modeler approval, the bot updates the domain model. Evaluations show the bot achieves median F1-scores of 86% for Found Configurations, 91% for Offered Suggestions and 90% for Updated Models, with a median processing time of 55.5 ms.

In [118], the authors employ NLP techniques and ML algorithms to automatically extract and rank the requirements terms to support high-level feature modeling. For that, they propose an automatic framework composed of a noun phrase-identification technique for requirements terms extraction and TextRank combined with semantic similarity for terms ranking. The final ranked terms are organized as a hierarchy, which can be used to help name elements when performing feature modeling [118]. In the quantitative evaluation, the proposed extraction method performs better than three baseline methods in recall with comparable precision. Their adapted TextRank algorithm can rank more relevant terms at the top positions in terms of average precision compared with most baselines [118].

Feature models (FMs) provide a visual abstraction of the variation points in the analysis of Software Product Lines (SPL), which comprise a family of related software products. FMs can be manually created by domain experts or extracted (semi-)automatically from textual documents such as product descriptions or requirements specifications [119]. In [119], a method to quantify and visualize whether the elements in an FM (features and relationships) conform to the information available in a set of specification documents is proposed. Both the correctness (choice of representative elements) and completeness (no missing elements) of the FM are considered.

Requirements traceability ensures that the developed system fulfils all requirements and prevents failures. For safety-critical systems, traceability is mandatory to ensure that the system is implemented correctly. Establishing and maintaining trace links are hard to achieve manually in today's complex systems [120]. To address this, the authors in [120] extend Siemens' Application Lifecycle Management (ALM) toolchain with an AI-powered module that automatically establishes bi-directional trace links between natural language requirements and various design artifacts (Capital™ (https://resources.sw.siemens.com/en-US/white-paper-ee-systems-development-flow/), AUTOSAR (https://www.autosar.org/standards/classic-platform), SysML (https://sysml.org), UML (https://www.uml.org), Arcadia (https://mbse-capella.org/arcadia.html)).

They demonstrate its integration and trace-linking use cases across system, software and hardware models.

The authors of [43] highlight that extracting information from SRS documents is key for SPL development, yet simple classifiers struggle with complex organizational contexts and inter-dependencies. To address this, they propose a Deep Learning, non-exclusive classification framework for functional requirements: documents are vectorized using both Word2Vec and FastText embeddings (retrained on AUTOSAR enterprise data and compared to public models) and then fed into a CNN. This approach outperforms traditional multi-class methods in capturing overlapping requirement categories.

The problems associated with the requirement analysis and class modeling can be overcome by the appropriate employment of ML. In [121], the authors present READ (Requirement Engineering Analysis Design), a Python-based system that uses NLP techniques and a domain ontology to automatically generate UML class diagrams, including class names, attributes, methods and relationships from English requirements text. Evaluated on

publicly available benchmarks, READ outperforms existing object-oriented design tools in accuracy and completeness.

The research reported in [122] applies NB classifiers, multinomial and Gaussian, over different SRS documents and classifies the software requirement entities (actors and use cases) using ML-based methods. The study used SRS documents of 28 different systems and labels for the entities 'Actor' and 'Use Case' were defined. MNB is a popular classifier because of its computational efficiency and relatively good predictive performance [122]. Several classifiers were tried out. The MNB recognizes actors and use cases with an accuracy of 91%. Actors and use cases can be extracted with high accuracy from the SRS documents using MNB, which then can be used for plotting the use case diagram of the system [122]. Automated UML model generation approaches have a very prominent role in an agile development environment where requirements change frequently.

Existing NLP approaches for processing requirements documents are often limited to specific model types, such as domain or process models and fail to reflect real-world requirements. To address this, in [123] a conceptual-level preprocessing pipeline is proposed, for automatically generating UML class, activity and use case models. The pipeline consists of three steps [123]: (1) entity-based extractive summarization to highlight key requirement sections, (2) rule-based bucketing to categorize sentences for different UML models and (3) sequence labeling to identify classes and attributes for class modeling. Since labeled datasets for this task are scarce, the authors have labeled the widely used PURE dataset, by tagging classes and attributes within the texts, on a word level, to train their supervised ML model [123].

In [124], a named entity-recognition method called SyAcUcNER (System Actor Use Case Named Entity Recognizer) is presented, with the goal of extracting the system, actor and use case entities from unstructured English descriptions of user requirements for the software. SyAcUcNER is a specialized Named Entity Recognizer (NER) pipeline for requirements text that extracts system, actor and use case entities. It applies semantic role labeling to tag words, then uses an SVM classifier (via WEKA) to assign domain-specific semantic labels. Evaluated on mixed-source corpora, SyAcUcNER achieved 76.2% precision, 76% recall and a 72.1% F-measure [124].

The process of testing industrial systems, integrating highly configurable safety-critical components, is highly time consuming and may require associations between product features and requirements demanded by customers [125]. ML was used to help engineers in this task, by automating the extraction of associations between features and requirements. However, when requirements are written in NL, several additional difficulties arise. In [125], an NLP-based model, called SRXCRM (System Requirement eXtraction with Chunking and Rule Mining), is presented, which is able to extract and associate components from product design specifications and customer requirements, written in NL, of safety-critical systems. The model has a weight association rule mining framework that defines associations between components, generating visualizations that can help engineers in the prioritization of the most impactful features [125]. Preliminary results show that SRXCRM can extract such associations and visualizations [125].

ReqVec is a semantic vector representation for functional requirements that encodes key dimensions, such as main actor, main action and affected element, to support tasks like dependency detection and categorization [126]. ReqVec's semantic vector is built in three steps: (1) lexical and syntactic analysis of requirement text; (2) computing semantic-dimension embeddings via a word-classifier and Word2Vec; and (3) assembling the final ReqVec from those dimension vectors. In experiments, ReqVec achieved a 0.92 F-measure for identifying related requirements and 0.88 F-measure for categorization.

In [127], the authors develop an approach to extract domain models from problem descriptions written in NL by combining rules based on NLP with ML. First, they present an automated approach with an accuracy of extracted domain models higher than existing approaches. In addition, the approach generates trace links for each model element of a domain model. These trace links enable novice modelers to execute queries on the extracted domain models to gain insights into the modeling decisions taken for improving their modeling skills [127]. Preliminary results are positive.

Use case analysis is a graphical depiction used to explain the interaction between the user and the system for the given user's task and it also denotes the extension/dependency of one use case to another. It is often used to identify, clarify and categorize system requirements. Generating use cases from inherently ambiguous NL descriptions of requirements may be hard work that can be automated using data-driven techniques [128]. The work in [128] presents an initial approach for the automated identification of use case names and actor names from the textual requirements specification using an NLP technique called Name Entity Recognition (NER) and extracts the named entity mentions in unstructured texts into predefined categories such as person names, organizations, locations, etc.

Model-driven engineering relies on transforming informal requirements into formal domain models, but automatically extracting relationships is hard because key patterns are often buried in sentence context. To address this, [129] groups sentences around identified domain concepts and applies a BiLSTM network to detect the hidden relationships and patterns among them. A subsequent classification step then categorizes these relationships. Preliminary experiments show the method effectively uncovers domain links, making it a promising direction for reducing the manual effort in domain modeling.

In [130], a domain modeling bot called DoMoBOT is introduced and implemented in the form of a web-based prototype. According to the authors, DoMoBOT automatically extracts a domain model from a problem description written in NL with an accuracy higher than existing approaches. The bot also enables modelers to update a part of the extracted domain model and, in response, the bot reconfigures the other parts of the domain model proactively. To improve the accuracy of extracted domain models, techniques of NLP and ML are combined [130].

The work in [131] proposes an Automated E-R Diagram Generation (AGER) system that can generate an E-R diagram from a given text in natural language. The AGER system parses an input text in natural language, semantically analyzes it and internally uses some domain-specific databases and PoS tagging to detect entity and relations from the given passage and builds a graph that represents the E-R diagram [131]. During a project's requirements analysis phase, software engineers often engage in discussions with clients about the intended use cases. The conclusion of this process may yield a comprehensive E-R diagram, which serves as the blueprint for implementing and materializing the database relationships in later stages. The AGER system is aimed at assisting in creating E-R diagrams directly from a client's requirements in natural language [131].

In [132], the authors propose an argumentation-based CrowdRE method that models fragmented online user feedback as structured user argumentation graphs. By applying abstract, bipolar and coalition-based meta-argumentation frameworks, the approach extracts features and issues along with their supporting and opposing arguments. Automated algorithms classify comments into rationale elements and identify conflict-free claims, demonstrating effective detection of features, issues and their associated arguments.

Some challenges have arisen in processing and analyzing user data for conversion into UML diagrams [133]. In response, the work in [133] introduces a novel approach that primarily aims to improve accuracy, shorten the time required to generate use cases from natural language descriptions and address shortcomings in current technologies. The

authors' goal is to create a smart, precise system that not only saves time but also enhances user trust in the software.

In [134], the authors propose an approach in which users provide exemplary behavioral descriptions rather than explicit requirements. They reduce the problem of synthesizing a requirements specification from examples to one of grammatical inference, applying an active coevolutionary learning approach [134]. While such an approach typically requires numerous user feedback queries, the authors extend active learning by incorporating multiple oracles, known as proactive learning [134]. In this framework, the "user oracle" supplies input from the user and the "knowledge oracle" provides formalized domain knowledge. Their two-oracle method, called the "first apply knowledge then query" (FAKT/Q) algorithm, is benchmarked against standard active learning, resulting in fewer required user queries and a faster inference process.

The work in [105], addressed in Section 5.2.3, also proposes a method, combining NLP and ML, to automate extracting semantic elements for software requirement-to-test mapping. It formalizes recommendation reviews, enhancing traceability, reliability and efficiency throughout the SDLC.

The paper in [135] summarizes the approach of the OpenReq-DD dependency-detection tool developed at the OpenReq project, which allows an automatic requirement dependency-detection approach. The core of this proposal is based on an ontology that defines dependency relations between specific terminologies related to the domain of the requirements. Using this information, it is possible to apply NLP techniques to extract meaning from these requirements and relations and ML techniques to apply conceptual clustering, with the major purpose of classifying these requirements into the defined ontology [135].

Requirement dependencies affect many activities in the SDLC and are the basis for various software-development decisions. Requirements dependency extraction is, however, an error-prone and a cognitively and computationally complex problem, since most of the requirements are documented in natural language [136]. A two-stage approach is proposed in [136] to extract requirements dependencies using NLP and Weakly supervised learning (WSL). In the first stage, binary dependencies (basic dependent/independent relationships) are identified and in the second stage, these are analyzed to determine the specific dependency type. An initial evaluation on the PURE dataset, using RF, SVM and NB was conducted. These three machine learners showed similar accuracy levels, although SVM required extra parameter tuning. The accuracy was further improved by applying weakly supervised learning to generate pseudo-annotations for unlabeled data [136]. The authors have defined a research agenda for assessing the use of their approach in different domains. To increase the semantic foundations, they intend to use evolving ontologies.

In [137], the authors present an approach to automatically identify requirement dependencies of type "requires" by using supervised classification techniques. The results indicate that the implemented approach can detect potential "requires" dependencies between requirements (formulated on a textual level). The approach was evaluated on a test dataset and the conclusion is that it is possible to identify requirement dependencies with a high prediction quality. The proposed system was trained and tested with different classifiers such as NB, Linear SVM, KNN and RF. RF classifiers correctly predicted dependencies with an F1-score of 82%.

Ignoring requirements inter-dependencies can adversely impact the design, development and testing of software products. In [138], a proposal addressing three main challenges is made: (1) NLP is studied to automatically extract dependencies from textual documents. Verb classifiers are used to automate elicitation and analysis of different types of dependencies (e.g.: requires, coupling). (2) Representation and maintenance of changing

requirement dependencies from designing graph theoretic algorithms is explored. (3) The process of providing recommendations of dependencies is studied. The results, still preliminary, are aimed at assisting project managers to evaluate the impact of inter-dependencies and make effective decisions in the software-development life cycle [138].

Many supervised learning methods have been applied to requirements traceability recovery (RTR), yet their performance remains unsatisfactory, prompting the need for more effective models. In [139], a new cascade deep forest model for RTR, called DF4RT (Deep Forest for Requirements Traceability), is proposed with a novel composition aimed at enhancing performance. The model integrates three feature-representation methods: information retrieval (IR), query quality (QQ) and distance metrics [139]. Additionally, it employs a layer-by-layer training approach to harness the benefits of DL while incorporating IR, QQ and distance features to promote input diversity and enhance model robustness [139]. DF4RT was evaluated on four open source projects and compared against nine state-of-the-art tracing approaches, showing average improvements of 94% in precision, 58% in recall and 72% in F-measure [139]. The proposed approach is effective for RTR with good interpretability, few parameters and good performance in small-scale data.

The traceability links between requirements and code are fundamental in supporting change management and software maintenance. Automatic trace retrieval can be performed via various tools such as information retrieval or ML techniques [140]. These tools have, however, a low-precision problem, which is primarily caused by the term mismatches across documents to be traced. The study in [140] proposes an approach that addresses the term mismatch problem to obtain the greatest improvements in the trace-retrieval accuracy. The approach uses clustering in the automated trace-retrieval process and, in an experimental evaluation against previous benchmarks, it showed results that allow one to conclude that the approach improves the trace-retrieval precision [140].

Traceability Link Recovery (TLR) was a topic of interest for many years within the software-engineering community and recently it has gained even more attention from both fundamental and applied research. However, there remains a significant gap between industry needs and the academic solutions proposed [141]. The work in [141] proposes an approach called Evolutionary Learning to Rank for Traceability Link Recovery (TLR-ELtoR), which combines evolutionary computation and ML techniques to recover traceability links between requirements and models by generating a ranked list of model fragments capable of fulfilling the requirement. TLR-ELtoR was evaluated in a real-world railway domain case study and compared against five TLR approaches (Information Retrieval, Linguistic Rule-based, Feedforward Neural Network, Recurrent Neural Network and Learning to Rank). The results demonstrate that TLR-ELtoR achieved the best performance on most indicators, with a mean precision of 59.91%, recall of 78.95%, a combined F-measure of 62.50% and an MCC value of 0.64 [141].

Many information retrieval-based approaches have been proposed to automatically recover software requirements traceability links. However, such approaches typically calculate textual similarities among software artifacts without considering specific features of different software artifacts, leading to less accurate results [142]. In [142], the authors present a hybrid method for recovering requirements traceability links by combining ML with logical reasoning to analyze both use case and code features. The approach first extracts semantic features from use cases and code, which are then used to train a classifier through supervised learning. Simultaneously, it examines the structural aspects of code to incrementally uncover traceability links using a set of defined reasoning rules. Experiments comparing this method to state-of-the-art techniques show that the proposed approach outperforms existing methods.

Supervised automated solutions to generate trace links use ML or Deep Learning techniques, but require large labeled datasets to train an effective model. Unsupervised solutions as word-embedding approaches can generate links by capturing the semantic meaning of artifacts and are gaining more attention. Despite that, the authors in [143] argue that, besides the semantic information, the sequential information of terms in the artifacts would provide additional assistance for building accurate links. In that sense, they propose an unsupervised requirements traceability approach (named S2Trace) which learns the sequential semantics of software artifacts to generate the trace links. Its core idea is to mine the sequential patterns and use them to learn the document-embedding representation. Five public datasets were evaluated and results show that the proposed approach outperforms three typical baselines. The modeling of sequential information in [143] provides new insights into the unsupervised traceability solutions and the improvement in the traceability accuracy further proves the usefulness of the sequential information.

5.2.5. Supporting Requirements Management and Validation and Project Management

Requirements prioritization, assessing risk, namely the impact of requirements' change requests on the project outcomes, allocating requirement to software versions, among other activities, are important topics in supporting project management and requirements management and validation. Table 6 lists the main NLP and ML-based approaches for supporting requirements management and validation and project management. These approaches, together with the reviewed literature references that use them, are addressed in this section.

Table 6. Main ML-based approaches to supporting requirements management and validation and project management.

RE Category	RE Activity	Main Approaches Used for NLP and Feature Extraction from NL Text, and for Dataset Preparation, and Main ML Approaches Used for Achieving the RE Activity
Supporting Requirements Management and Validation, and Project Management	Requirements Prioritization	Adam algorithm; ARPT (Automated Requirement Prioritization Technique); Decision Tree, Random Forest, and K-Nearest Neighbors; NLP + ML; Combination of NLP techniques and Machine Learning algorithms; Adapted genetic K-means algorithm for software requirements engineering (GKA-RE), which automatically identifies the optimal number of clusters by dynamically readjusting initial seeds for improved quality; AI Task Allocation tool (ATA'); Tree-Family Machine Learning (TF-ML); Credal Decision Tree (CDT).
	Allocating requirements to software versions based on development time, priority, and dependencies; Predicting whether a software feature will be completed within its planned iteration	
	Project Management Risks Assessment / Req. Change Impact analysis on other requirements and on planned test cases	

The study in [144] proposes an optimization algorithm that aims to select features to give meaningful information about requirements. These features can be used to train a model for prioritizing requirements. The study examines how optimization algorithms select features and assign requirement priorities. It reveals that the Adam algorithm struggles with accurately prioritizing requirements due to the sparse matrix generated for the text dataset and high computational cost and it fails to consider requirement dependencies. To address these issues, the paper introduces the Automated Requirement Prioritization Technique (ARPT) [144]. Compared to the Adam algorithm, ARPT achieves a much lower mean squared error of 1.29 versus 6.36 and its execution time is only 1.99 ms compared to 3380 ms [144].

Prioritizing software requirements for a release is complex, especially when requested features exceed development capacity. This challenge grows with multiple product and business lines, involving diverse stakeholders. The study in [145] applies ML to optimize release planning by analyzing key parameters influencing requirement inclusion. Five models were tested using accuracy, F1-score and K-Fold Cross Validation. While most models achieved 80% accuracy, further investigation led to improved results. DT, RF and

K-Nearest Neighbors performed best, with optimized RF reaching 100% accuracy in some metrics but at high computational cost. Future work may refine other models through hyperparameter tuning.

The authors in [146] found thirteen Requirements Prioritization (RP) methods applying AI techniques such as ML or genetic algorithms, 38% of which seek to improve the scalability problem, whereas 15% of them aim to improve the lack of automation issues along the RP process. In order to address the issues of scalability and lack of automation in RP, the study in [146] proposes a semi-automatic multiple-criteria prioritization method for functional and non-functional requirements of software projects developed within the software product lines paradigm. The proposed RP method is based on a combination of NLP techniques and ML algorithms and empirical studies will be carried out with real web-based geographic information systems (GISs) for its validation [146].

Similar to standard systems, the identification and prioritization of the user needs are relevant to the software quality and challenging in SPL due to common requirements, increasing dependencies and diversity of stakeholders involved [147]. As the prioritization process might become impractical when the number of derived products grows, recently there was an exponential growth in the use of AI techniques in different areas of RE. In [147], a semi-automatic multiple-criteria-prioritization process is proposed, for functional and non-functional requirements (FR/NFR) of software projects developed within the SPL paradigm for reducing stakeholder participation.

The clustering stakeholder problem for system requirements selection and prioritization is considered inheritance in the area of requirements engineering. In [148], the authors apply clustering techniques from marketing segmentation to determine the optimal number of stakeholder groups. They introduce an adapted genetic K-means algorithm for software requirements engineering (GKA-RE) that automatically identifies the optimal number of clusters by dynamically readjusting initial seeds for improved quality. The method was tested on two RALIC system requirements datasets using various evaluation metrics and its performance was compared with that of the standard K-means approach. The experimental results indicate the superiority of GKA-RE over the K-means approach in obtaining higher values of evaluation metrics [148].

There are many fundamental prioritization techniques available to prioritize software requirements. In automating various tasks of software engineering, ML has shown useful positive impacts. The work in [149] discusses the various algorithms used to classify and prioritize software requirements. The results in terms of performance, scalability and accuracy from different studies are contradictory in nature due to variations in research methodologies and the type of dataset used. Based on the literature survey conducted, the authors have proposed a new architecture that uses both types of datasets, i.e., SRS and user text reviews to create a generalized model. The proposed architecture attempts to extract features which can be used to train the model using ML algorithms. The ML algorithms for classifying and prioritizing software requirements will be developed and assessed based on performance, scalability and accuracy.

The study in [77], addressed above, explores a speech act-based linguistic technique to analyze online discussions and extract requirements-relevant information, also for RP. By applying NLP and ML, user reviews from various sources are filtered and classified into categories like feature requests and bug reports. The authors refine their prior speech act analysis approach and evaluate it on datasets from an open source and an industrial project. The method achieves F-scores of 0.81 and 0.84 in classifying messages as feature/enhancement or other. Results indicate a correlation between certain speech acts and issue priority, supporting automated RP.

The work in [150] introduces the AI Task Allocation tool (ATA') for distributing software requirements across different versions using an AI planning approach. The authors propose a model, expressed in an AI planning language, that represents a software project with a set of requirements and one or more development teams. The generated plan assigns requirements based on development time, priority levels and dependency relationships [150]. A case study was conducted to evaluate the ATA' tool and preliminary results show that the generated plans allocate requirements according to the specified criteria. These findings suggest that ATA' can effectively support the planning of incremental development projects by facilitating the allocation of requirements among teams.

In [151], the authors address the challenge of predicting whether a high-level software requirement will be completed within its planned iteration, a key factor in release planning. While prior research focused on predicting low-level tasks like bug fixes, this study analyzes iteration changes across three large IBM projects, revealing that up to 54% of high-level requirements miss their planned iteration. To tackle this, the authors develop and evaluate an ML model using 29 features derived from prior research, developer interviews and domain knowledge. The model, tested at four requirement lifecycle stages, achieves up to 100% precision. Feature importance analysis shows that some factors are project-specific or stage-specific, while others, such as time remaining in the iteration and requirement creator, consistently influence predictions.

The success of NL interfaces in interpreting and responding to requests is, to a large extent, dependent on rich underlying ontologies and conceptual models that understand the technical or domain-specific vocabulary of different users [152]. The effective use of NL interfaces in Software Engineering (SE) requires dedicated ontology models for software-related terms and concepts [152]. Although there are many SE glossaries, these are often incomplete and focus on specific sub-fields without capturing associations between terms, limiting their utility for NL tasks. To address this, the authors in [152] propose an approach that starts with existing glossaries and their defined associations and uses ML to dynamically identify and document additional relationships between terms. The resulting semantic network is used to interpret NL queries in the SE domain and is enhanced with user feedback. Evaluated in the sub-domain of Agile Software Development, focusing on requirements-related queries, the approach shows that the semantic network significantly improves the NL interface's ability to interpret and execute user queries.

Risk prediction in the SDLC is vital for project success. In [153], the authors evaluate ten tree-family ML techniques on a requirements risk dataset and find that the Credal Decision Tree (CDT) performs best. In 10-fold cross-validation, CDT achieves a Mean Absolute Error (MAE) of 0.0126, Root Mean Squared Error (RMSE) of 0.0888, Relative Absolute Error (RAE) of 4.50%, Root Relative Squared Error (RRSE) of 23.74% and 98% accuracy (with recall, F-measure at 0.98 and Matthews Correlation Coefficient (MCC) at 0.975). The study recommends CDT for requirements risk prediction and suggests that future work tackle class imbalance, feature selection and Ensemble Methods.

Software requirements Change Impact Analysis (CIA) is crucial in RE since changes are inevitable. When a requirement change is requested, its effects on all software artifacts must be evaluated to decide whether to accept or reject it. The research in [154] proposes a prediction model using a combination of ML and NLP techniques to forecast the impact of a requirement change on other requirements in the SRS document. This ongoing work will evaluate the proposed model with relevant datasets for accuracy and performance. The resulting tool may be used by project managers to perform automated change impact analysis and make informed decisions about requirement change requests [154].

Incomplete requirements elicitation and elaboration often leads to functional requirements changes, which need to be controlled. Each function in a functional requirement

includes its name, description, input/output specifications and error messages. These specifications detail input and output names, data types and constraints and may or may not be linked to the database schema. Therefore, when changes occur in inputs or outputs that impact the database schema, both the schema and the corresponding test cases can be affected. The work in [155] presents an approach for analyzing the impact on test cases when inputs or outputs of functional requirements are modified. This method provides a structured change process to manage updates to functional requirements, test cases and the database schema and it also includes a rollback feature to reverse changes if necessary.

6. Analysis and Discussion

In this section, the reviewed literature's results, presented in the previous section as belonging to five categories of RE tasks, is further discussed.

The main discussion topics aim to provide the answers to the previously stated research questions, namely:

RQ1 Which Requirements Engineering activities take advantage of the use of Artificial Intelligence techniques?

RQ2 Which Artificial Intelligence techniques are most used in each Requirements Engineering activity?

RQ3 Which Artificial Intelligence techniques have the best results in each Requirements Engineering activity?

Figure 4 shows the percentage of references in the reviewed literature for each of the identified categories. One may see that 26% of the works reviewed were categorized as "Classification of Requirements according to their Functional/Non-Functional nature" and another 26% as "Extracting Knowledge from Requirements". These are the RE activities with most AI-based proposals for automation or semi-automation of tasks. Then, there is "Improving the Quality of Requirements and Software", with 24% of the articles reviewed, "Supporting Requirements Elicitation" with 15%, and "Supporting Requirements Management and Validation and Project Management" with 9%. Recall that some references may appear in more than one category, in the cases they address several RE tasks.

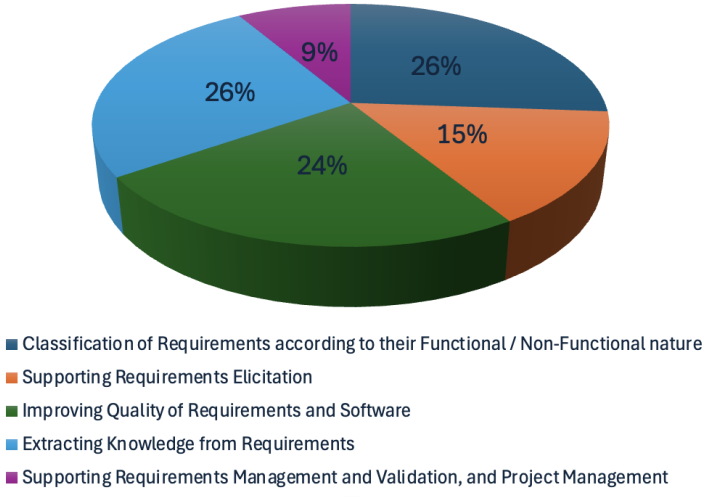

Figure 4. Categories of RE tasks found in the reviewed literature.

Most of the studied approaches have used for text preprocessing and feature extraction, at the early phase of dealing with requirements written in NL, techniques for tokenization, normalization, lemmatization and vectorization, in order to convert raw text into structured

numerical feature vector representations. Examples of such techniques are bag of words (BoW) [40], which quantifies the frequency of words in text documents, yielding a vector where each dimension corresponds to the frequency of a word, and TF-IDF [36,39,59], which weighs the word frequencies by how unique or important they are across documents. With a standard BoW model, any word can have the highest term frequency, regardless of its semantic relevance. TF-IDF, which is a variation of the BoW, accounts for the word's prevalence throughout every document in a text set or corpus of documents. Once the requirements text is preprocessed and transformed into numerical feature vectors, a classification algorithm can be applied.

Besides BoW and TF-IDF, other used NLP techniques for data preprocessing and extracting syntactic and semantic features from natural language text in preparation for any of the mentioned RE activities are Word2Vec, FastText, Doc2Vec and GloVe, which are able to capture semantic relationships between words. Tools like spaCy and NLTK also facilitate these steps. These techniques can be combined in a pipeline to convert raw NL text into structured numerical representations that an ML classifier can then use to learn patterns and make predictions.

To improve the accuracy and efficiency in classifying FR, NFR and other specialized NFR requirements, after addressing the challenges posed by unstructured natural language data, many studies propose traditional ML algorithms, such as NB, DT, SVM and Logistic Regression. The main ML approaches to classifying requirements as FR, NFR or non-requirements and for sub-categorizing NFR into their subcategories, are DT, SVM, NB, RF, KNN, among others (refer to Table 2).

To identify preconditions and postconditions within requirements and categorize them into groups (e.g., "none" vs. "both", indicating the presence or absence of preconditions and postconditions in requirements), for test case generation, the used technique was NB with text preprocessing using libraries such as Scikit-learn and NLTK [104].

Supervised classifiers were also used to automatically assess user stories against criteria (e.g., testable and valuable from the INVEST checklist) [106] and for automatically labeling requirements into testing levels (e.g., 'Integration Test' vs. 'Software Test') [111].

The results also show that the selection of the feature-extraction techniques used after the NL text-preprocessing phase and before the ML classifiers entering into action is important, as the chosen techniques influence the performance of the classifiers. For instance, selecting TF-IDF before different ML algorithms for classifying software requirements, namely DT, RF, LR, NN, KNN and SVM, yielded better performance than using Word2Vec before the same algorithms [36].

Some researchers further enhance these models using Ensemble Methods such as Bagging (Bagged KNN, Bagged DT, Bagged NB), RF, Gradient Boosting, AdaBoost and majority voting ensembles. Some of the proposed ensemble techniques that seem to have yielded satisfactory results are Bagged KNN, Bagged DT and Bagged NB, which are Ensemble Methods where the respective classifiers (KNN, DT and NB) are trained on different bootstrapped subsets of the training data [46,53]. This process, known as bagging (bootstrap aggregation), involves building several models and obtaining several outputs before combining and aggregating their predictions to enhance overall performance.

Other proposed Ensemble Methods are Ensemble Grid Search classifier using five models (RF, MNB, GB, XGBoost, AdaBoost) [37] or Ensemble classifier combining five models (NB, SVM, DT, LR, SVC) [54]. These also work several solutions before aggregating results to obtain a better prediction. Ensemble Techniques, such as boosting (Adaptive Boosting, Gradient Boosting, Extreme Gradient Boosting) were also applied to identify fault-prone requirements [82].

Active learning approaches, based on strategies (e.g., uncertainty sampling) were used in the selection of the most informative data points for labeling, reducing manual effort in building high-quality training sets.

Supervised ML and Ensemble Methods, combining information retrieval, query quality and distance metrics using cascade deep forest models (DF4RT) for traceability [139], were also proposed to automate the generation of traceability links between requirements and other artifacts. Hybrid and unsupervised approaches use sequential semantics (S2Trace) to mine patterns in requirements for more accurate link generation; they were also used for the same purpose [143].

Multiple ML classifiers (e.g., DT, RF, KNN) were employed to predict requirement inclusion for releases [145,149]. Ensemble approaches and hyperparameter tuning further improve accuracy, addressing the complexity of prioritizing features when stakeholder demands exceed development capacity.

In [154], a combined NLP and ML model predicts how changes in requirements affect other parts of the SRS, supporting automated change impact analysis and aiding project managers in decision making.

NLP techniques for extracting structured information (e.g., function names, I/O specifications) from functional requirements were also used [155], followed by an ML-based approach to assess how changes affect test cases and database schemas, offering a structured change process and rollback features.

Some advanced Deep Learning approaches were also proposed, including RNN-based models (Bi-LSTM, Bi-GRU) often combined with self-attention mechanisms and transformer-based architectures (BERT, BERT-CNN, PRCBERT) [38,41,45,60]. These models are designed to better capture contextual and sequential information in requirements texts. RNN and variants, such as Bi-LSTM and Bi-GRU, capture sequential dependencies in requirements text and were also proposed for improving quality of requirements. Some of them incorporate self-attention mechanisms to better aggregate contextual information [38,60]. CNN was also used, sometimes in hybrid models (e.g., BERT-CNN), to extract task-specific features from preprocessed text [45].

More advanced Transformer-based models, such as BERT, GPT-3.5 and PRCBERT are integrated for tasks like sentence classification, prompt-based learning and even generating requirements, exploiting their contextual understanding. In fact, using Deep Learning transformer models such as BERT and GPT and several variations (e.g., PRCBERT) is also becoming more common in classifying requirements as FR, NFR or non-requirements, subcategorizing NFR into their subcategories [45,47]. These type of models are, however, more demanding in terms of training data and are more common for more complex RE activities, such as supporting requirements elicitation [66,74] or improving quality of requirements and software [85,89,95], as can be seen in Tables 3 and 4, respectively.

For identifying requirements' ambiguities and generating use case specifications to zero-shot classification, the most used models were BERT, GPT-3.5 and PRCBERT, due to their superior contextual understanding [64].

Other techniques, such as zero-shot learning, were also proposed to classify requirements without extensive labeled data.

Rasa-NLU and Rasa-Core open source frameworks were used for building conversational agents that can elicit requirements from stakeholders through natural language interactions [56]. The chatbot leverages an LSTM-based model for dialogue interpretation, followed by ML-based classification of the elicited requirements.

Large Language Models (LLMs), such as ChatGPT, were used for generating model elements from natural language, in requirements analysis and domain and design modeling tasks [58,85,116]. These, however, have shown some issues with traceability and consistency,

which indicate that human oversight remains necessary. To extract domain models from agile product backlogs, two approaches were attempted, namely, the use of LLMs with specialized NLP tools [116] and bot–modeler interaction models for updating domain models [117].

In [144], an optimization algorithm is used to select meaningful features from requirements. These features are used to train a model for prioritizing requirements. The Automated Requirement Prioritization Technique (ARPT) improves on the Adam algorithm by reducing error (lower mean squared error) and execution time.

GKA-RE, a genetic K-means algorithm, is applied in [148] to group stakeholders. By dynamically readjusting initial seeds, the technique determines the optimal number of clusters, aiding in selecting and prioritizing requirements based on stakeholder segmentation.

Specialized techniques for improving requirements quality have also been proposed.

Speech act analysis, an NLP technique for extracting linguistic features, combined with ML-based classification, has been used to analyze online discussions by categorizing messages according to their speech acts. This approach enables filtering and categorizing user reviews (e.g., as feature requests or bug reports) and even issue priorities [77].

Multimodal representations, autoencoder-based models that combine text and visual features, were investigated for analyzing user reviews when labeled data are scarce [69].

Clustering and hierarchical labeling techniques, for contextual word embeddings, were proposed to group similar requirements, for identifying duplicates or handling multi-granularity issues. To creatively generate candidate requirements, by applying perturbations to original requirement descriptions, Adversarial Example Generation was proposed [68].

Combining information from various sources in knowledge graphs (e.g., security practices) was also used to enhance requirements management.

Ontology-based and formal methods, such as BASAALT and FORM-L were proposed for formalizing requirements, by generating semantically precise and simulable models and support behavioral simulation, ensuring that ambiguities and inconsistencies are addressed [83].

Integrating ontologies with ML and optimization algorithms were proposed for creating a traceability matrix, relating requirements and test cases while incorporating real-world knowledge, thus achieving high dependability and stability [103].

To extract domain concepts and relationships to build domain vocabularies or ontologies, methods that integrate information entropy and the CCM method have been proposed [112]. These methods help to improve the automation of ontology construction, enhancing the precision of user requirements.

The Rule-Based Ontology Framework (ROF) was used to automatically generate requirements specifications and process models (BPMN, IEEE SRS) from elicitation outputs stored in an ontology [115].

In [152], ML is applied to extend existing SE glossaries, automatically identifying additional relationships between terms. This semantic network enhances natural language interfaces by improving domain understanding.

Other approaches used in improving requirements quality were Active Learning and AutoML, with techniques such as uncertainty sampling and tools such as TPOT [49]. These were used to optimize model performance and reduce manual labeling efforts. And also, Transfer Learning and Text Augmentation techniques (e.g., ULMFiT and back translation) were applied to improve classification performance, especially when data are limited [90].

Cosine similarity and standard deviation-based methods for selecting and improving training data quality were also used, in order to automatically generate test cases from requirements specifications with higher accuracy using fewer training data points [108].

Converting SRS data into formal representations using Information Extraction (IE) techniques, to automatically transform unstructured or semi-structured SRS text into structured data that can be converted into formal logic or model elements, was also proposed [114].

A combination of metamodel matching with transformation-by-example techniques, to accelerate model transformations from requirements by matching metamodels and analyzing consistency, was used in [113].

To extract and hierarchically organize key requirement terms, a technique that combines noun phrase identification with TextRank and semantic similarity to rank terms was used [118].

To extract domain-specific entities (e.g., system, actor, use case) from textual requirements, in [124] the SyAcUcNER approach was proposed. It uses semantic role labeling and SVM to identify entities, aiding UML diagram generation.

To identify and map associations between features and requirements, especially for safety-critical systems, SRXCRM was proposed. It applies weight association rule mining combined with NLP to visualize dependencies [125]. Additionally, techniques like ReqVec were used to generate semantic vector representations to capture relationships and dependencies among requirements [126].

In [150], an AI Task Allocation tool utilizes an AI planning language to distribute requirements across software versions based on development time, priority and dependency relationships, supporting incremental project planning.

In [151], an ML model predicts whether high-level requirements will be met within planned iterations by using a set of 29 features, improving release planning through precise predictions of requirement completion.

In [153], a requirement risk-prediction model uses various tree-family ML techniques, with the Credal Decision Tree (CDT) showing superior performance in terms of accuracy, error metrics and overall precision. This helps forecast risks early in the SDLC.

The following subsections look to answer the three posed research questions.

6.1. RQ1—Which Requirements Engineering Activities Take Advantage of the Use of Artificial Intelligence Techniques?

AI and ML techniques have been applied across virtually every RE activity, but certain tasks stand out as primary beneficiaries:

- Requirements Classification: Automatically distinguishing functional, non-functional and other requirement types;
- Requirements Prioritization: Ranking requirements by importance, risk or stakeholder value;
- Traceability (Link Generation and Refinement): Creating and refining links between requirements and other artifacts (design elements, test cases, code);
- Ambiguity Detection and Disambiguation: Identifying vague or conflicting language in natural language requirements;
- Model Generation: Translating requirements text into structured models (e.g., UML diagrams, domain ontologies);
- Validation and Verification: Checking requirements for correctness, completeness, consistency and other quality attributes;
- Change Impact Analysis: Predicting how modifications to one requirement affect other requirements or other RE artifacts (e.g., models, work time predictions);

6.2. RQ2—Which Artificial Intelligence Techniques Are Most Used in Each Requirements Engineering Activity?

The most commonly applied AI methods and text-processing steps, found in the literature for each of the RE activities identified in RQ1, are shown in Table 7.

Table 7. Most common AI/ML techniques and feature-extraction and -preprocessing approaches used in RE activities.

RE Activity	Common AI/ML Techniques	Feature Extraction & Preprocessing
Requirements Classification	Naïve Bayes (NB), Decision Trees (DT), Support Vector Machines (SVM), Random Forest (RF), K-Nearest Neighbors (KNN), Logistic Regression (LR)	Bag-of-Words (BoW), TF-IDF, Word2Vec, FastText, GloVe, Doc2Vec
Requirements Prioritization	SVM, DT, KNN, NB, LR, Multinomial NB (MNB), Ensemble Methods (Bagged KNN/DT/NB), Genetic K-means (GKA-RE), ARPT optimization	TF-IDF, Feature selection via optimization
Traceability	RF, DT, NB, Ensemble classifiers (e.g., cascade deep forest DF4RT), Sequential-semantics models (S2Trace), Hybrid approaches	TF-IDF, Embeddings, Knowledge-graph features
Ambiguity Detection & Disambiguation	Transformer models (BERT, PRCBERT, GPT-3.5), Semi-automated NLP tools	Contextual embeddings (BERT), Lexical analyses
Model Generation	Heuristic Rule-based methods, Ontology-based frameworks (ROF), RNNs (Bi-LSTM, Bi-GRU), CNN, BERT-CNN hybrids	Dependency parses, Semantic Role Labeling
Validation & Verification	ML classifiers (NB, SVM, RF), Knowledge-oriented methods, Formal models, Prototyping tools	TF-IDF, Syntactic Features
Change Impact Analysis	Combined NLP+ML (e.g., ARPT-Adam, cascade models), Optimization algorithms, Feature-impact prediction models	TF-IDF, Domain-specific information extraction

6.3. RQ3—Which Artificial Intelligence Techniques Have the Best Results in Each Requirements Engineering Activity?

While traditional ML pipelines remain effective for well-defined tasks, such as requirements classification and prioritization, Ensemble Methods and transformer-based Deep Learning show the greatest promise for complex RE activities requiring improved NL understanding and large context modeling.

The AI techniques found in the literature that yielded the best results for each Requirements Engineering activity identified in RQ1 are:

- Requirements Classification and Prioritization: Pipelines using TF-IDF feature vectors followed by ensemble classifiers (e.g., Bagged DT/RF, Gradient Boosting) consistently outperform Word2Vec based setups in accuracy and robustness [36]. Hybrid optimization methods, such as the ARPT technique improving Adam's convergence [144] and GKA RE for stakeholder clustering [148], reduce error and speed up prioritization.
- Traceability: Cascade deep forest (DF4RT) and ensemble grid search classifiers achieve high precision/recall when linking requirements to artifacts [139], while sequential semantics mining (S2Trace) enhances link generation in unsupervised scenarios [143].
- Ambiguity Detection: Transformer-based models, such as BERT, GPT 3.5 and PRCBERT, offer superior contextual understanding, cutting ambiguity-detection errors by up to 20% over classic ML approaches [64].
- Model Generation: Heuristic rule-based frameworks for generating UML diagrams remain highly explainable [28]. When richer context is needed, semantic role labeling combined with SVM (e.g., SyAcUcNER) improves entity extraction and diagram accuracy [124]. RNNs with self attention (Bi LSTM + attention) also excel at mapping free text to structured model elements when ample training data are available [38,60].
- Validation and Verification: Knowledge-oriented ML methods, particularly Naïve Bayes and SVM, paired with formal modeling tools yield the most reliable defect-detection rates in requirements validation [30].

- Change Impact Analysis: Integrated NLP + ML frameworks, such as the combined model in [154], merge feature impact prediction with optimization algorithms to deliver the most accurate forecasts of requirement ripple effects.

7. Conclusions

In this article, the main AI techniques for Requirements Engineering, from the last five years, were comprehensively reviewed and grouped in five RE task categories: classification of requirements; supporting requirements elicitation; improving requirements and software quality; extracting knowledge from requirements; and supporting requirements management and validation and project management.

From the review, one can conclude that for supporting requirements classification, for identifying FR/NFR or for other purposes, the integration of traditional ML classifiers (NB, SVM, DT, RF, KNN) with effective NLP feature-extraction techniques (BoW, TF-IDF, word embeddings) are the most used approaches.

Conversational agents and chatbots, built with frameworks like Rasa, combined with active learning strategies, were proposed to facilitate the elicitation of requirements directly from stakeholders. This approach not only reduces manual effort but also minimizes errors by dynamically capturing and classifying requirements during stakeholder interactions.

For improving the quality of requirements, techniques such as the BASAALT/FORM-L approach and NLP4ReF leverage both rule-based and ML methods to formalize natural language requirements. By transforming unstructured text into precise, simulable models, these techniques support behavioral simulation and early detection of ambiguities, inconsistencies and other quality issues, thereby ensuring that the requirements accurately reflect stakeholder needs.

The use of NLP techniques like Named Entity Recognition (NER), text chunking and rule-based extraction, along with ML-based classification, has enabled the automated identification of domain-specific entities, such as classes, attributes and relationships. This capability supports the generation of UML diagrams and domain models, thereby improving traceability and facilitating efficient system design. And the combination of NLP and ML techniques, by incorporating methods for traceability link recovery, risk prediction using tree-based ML approaches and AI planning for requirement allocation was used to support requirements management. These methods support human decision-making across the SDLC by ensuring accurate change impact analysis, prioritization and alignment with project goals.

The next subsections resume challenges and open problems, including ethical implications of using AI in requirements engineering and propose future research directions.

7.1. Challenges and Open Problems

The main challenges and open problems that result from the literature analysis are:

- Data Quality and Availability: Many requirement datasets lack full labels (e.g., FR vs. NFR) or use inconsistent labeling schemes, making supervised training difficult. Also, organizations often keep their requirements documents internal, which, while understandable, creates proprietary data silos that limit the public availability of corpora and make reproducibility and benchmarking difficult.
- Ambiguity and Natural Language Complexity: Human language can express the same intent in many ways, so simple keyword methods (BoW, TF-IDF) miss context that transformers can catch, but even these can misinterpret implied stakeholder intent. Requirements often refer to concepts introduced before in a document and capturing long-range concepts' dependencies in long requirements documents is nontrivial.

- Feature Representation Trade-offs: BoW/TF-IDF give interpretable but high-dimensional sparse vectors, while embeddings (Word2Vec, BERT) are dense and semantically rich but opaque. Selecting or fusing these representations for optimal classifier performance in a given RE task remains undone. General purpose embeddings may not capture a specific domain concepts. Retraining or adapting embeddings for specific domains increases complexity.
- Model Explainability and Transparency: Ensemble Methods and deep networks can be highly accurate but offer little insight into why a requirement was classified a certain way or prioritized next. Stakeholders need clear/interpretable justification for predictions if they are to rely on automated suggestions.
- Generalization and Domain Adaptation: Models trained on one domain or corpus often underperform elsewhere. For instance, a model trained on finance domain requirements may perform poorly on healthcare or automotive texts. While fine tuning BERT helps, we still lack systematic methods to transfer small amounts of domain-specific data into robust RE models.
- Integration into Development Workflows: Seamless tool chain integration and automation of downstream SDLC steps is still a problem. AI models often run as separate services; tight integration with common RE tools (JIRA, IBM DOORS, GitHub Issues) is scarce. Also, in agile contexts, requirements change daily; models must process updates incrementally, ideally in real time, without full retraining.
- Traceability Link Quality: Precision vs. Recall Trade-off in automated link generation. High recall yields many false positives (spurious links), whereas high precision misses valid links. Striking the right balance for a particular project remains an open task. Automatic Link refinement (pruning or clustering similar links) to avoid over- or under-tracing artifacts is still underexplored.
- Scalability and Performance: Computational costs are high. Transformer models like BERT are resource intensive, making them slow on large backlogs. Few solutions exist for incremental learning, updating models on the fly, as new requirement data arrive.
- Evaluation and Benchmarking: Varying metrics and datasets make cross-study comparison difficult. Different studies use F1, accuracy, MCC or custom cost-based metrics, making comparison difficult. Also, there is a lack of standardized benchmarks for comparing RE-focused AI techniques.

Requirements engineering per se raises several ethical questions [157]. The introduction of AI into requirements engineering may exacerbate some of these issues and raise several other ethical considerations [158]. Table 8 lists the main ethical concerns raised by using AI in requirements engineering.

Table 8. Ethical questions raised by using AI in requirements engineering activities.

Key Ethical Concerns	Explanation
Bias and Fairness	Training datasets reflect historical biases (e.g., under-serving certain user groups) and AI may prioritize or classify requirements in ways that perpetuate inequity. Automated prioritization risks amplifying the needs of louder or more active stakeholders (whose feedback dominates the data), marginalizing quieter or less technical voices.
Transparency and Accountability	Deep Learning or ensemble models often lack clear explanations, making it hard to justify why a requirement was flagged, deprioritized or linked. Another issue has to do with decision ownership. When AI makes suggestions, who bears responsibility for errors?

Table 8. *Cont.*

Key Ethical Concerns	Explanation
Privacy and Confidentiality	Requirements documents can contain proprietary, personal or security-critical information. Feeding them into third-party AI services or cloud-based models risks leaks or unauthorized data exposure. In CrowdRE, scraping and analyzing user comments may inadvertently harvest personal data or violate users' expectations of privacy.
Over-Reliance and De-Skilling	Excessive dependence on AI suggestions can weaken engineers' requirement-analysis skills over time, reducing their ability to spot subtle domain issues or think critically. Teams may uncritically accept AI outputs, even when they are flawed, simply because "the tool suggested it."
Consent and Ethical Data Use	Using stakeholder inputs, such as chat logs or survey responses, to train or fine-tune models requires clear consent, especially if data is repurposed for unrelated RE tasks. Also, data collected for one purpose (e.g., feature requests) might be redeployed elsewhere without stakeholders' knowledge or approval.
Job Displacement and Workforce Impact	Automating routine tasks (e.g., classification, traceability) may shift or eliminate parts of requirements-engineer roles, demanding reskilling or raising concerns about job security. Organizations should plan for fair upskilling pathways rather than abrupt layoffs.
Security and Adversarial Manipulation	Malicious stakeholders could inject misleading or adversarial requirement descriptions to skew AI outputs (e.g., burying safety-critical requirements in noise). Model robustness requires that AI remains reliable under intentionally crafted or noisy inputs. This is essential for safety-critical domains.
Legal and Regulatory Compliance	If AI-driven requirement decisions lead to non-compliance (e.g., with GDPR), determining legal liability between tool vendors, integrators and engineering teams can be complex. Regulations may mandate clear records of how requirements were derived and prioritized, enabling audit trails. Opaque AI processes complicate compliance.

7.2. Future Research Directions

Based on the identified challenges and open problems, some ideas for future research are given below:

- Unified Benchmark Suites: Curating and publishing RE corpora/datasets with consistent labels for classification, prioritization, traceability, etc., along with baseline results and shared evaluation scripts, is a future research direction. This would accelerate progress by enabling fair comparison of new algorithms.
- Hybrid Models for Explainability: Combining symbolic rule engines or small decision trees with dense embeddings, so that each prediction can be traced to a rule or feature weight, would give stakeholders a transparent explanation for the prediction, without sacrificing deep models' accuracy.
- Domain Adaptive and Transfer Learning: To cut down expensive re-annotation efforts and improve domain performance, future research could work on developing systematic methods to adapt large pre-trained language models to new RE domains using minimal domain-specific data.
- Lightweight and Incremental Learning: Research online and continual learning algorithms that update models with each new requirement without retraining from scratch would keep models up to date with low computational overhead. This could be achieved by using techniques like parameter-efficient fine tuning, distillation or sparsity.
- Multimodal and Knowledge-Enhanced Approaches: Integrating text with UML sketches, prototypical screenshots and domain knowledge graphs (e.g., GDPR ontologies) in a unified model could leverage richer context, improving tasks like traceability and model generation.
- Human in the Loop and Active Learning: Building RE tools that identify the most uncertain or high impact requirements and query engineers for labels, refining the

model with minimal human effort, could maximize label efficiency and continuously improve model accuracy.

- End to End RE Automation Pipelines: Seamlessly chain AI components (e.g., elicitation chatbots, classification, prioritization, traceability and impact analysis) so that the output of one directly feeds the next component. This could reduce manual component chaining and accelerate the entire requirements engineering lifecycle.
- Robustness and Fairness in RE Models: To ensure equitable prioritization and reduce systemic errors, requirements datasets and trained models could be audited for biases and develop mitigation strategies.
- Efficient Transformer Variants: Adapting lightweight transformer architectures (ALBERT, DistilBERT, MobileBERT) specifically tuned for RE tasks or exploring retrieval augmented generation to minimize fine tuning could delivers much of BERT's power at a fraction of the computational cost.
- Empirical Studies and Industrial Adoption: Conducting large scale, real-world trials of AI-driven RE tools in diverse organizations and documenting ROI, usability and barriers to uptake would provide evidence for best practices, drive adoption and uncover new practical requirements for AI models.
- Ethical concerns: Carefully addressing ethical dimensions, through bias audits, explainable AI techniques, strict data governance, clear consent practices and human-in-the-loop oversight, will be crucial to responsibly deploying AI in requirements engineering.

Addressing the enumerated challenges and pursuing these suggested research directions can help us move closer to fully leveraging AI's potential to streamline, improve and even automate key aspects of requirements engineering.

Author Contributions: This research article is the result of research and findings of both authors, being the individual contributions as follows: Conceptualization, A.M.R.d.C.; methodology, A.M.R.d.C. and E.F.C.; investigation, A.M.R.d.C. and E.F.C.; resources, A.M.R.d.C.; formal analysis, A.M.R.d.C.; writing—original draft preparation, A.M.R.d.C. and E.F.C.; writing—review and editing, A.M.R.d.C. and E.F.C. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Conflicts of Interest: The authors declare no conflicts of interest.

Abbreviations

The following abbreviations are used in this manuscript:

ACM	Ambiguity Classification Model
AGER	Automated E-R Diagram Generation
AI	Artificial Intelligence
ALM	Application Lifecycle Management
ANN	Artificial Neural Networks
ASFR	Architecturally Significant Functional Requirement
ALBERT	A Lite BERT for Self-supervised Learning of Language Representations
BERT	Bidirectional Encoder Representations from Transformers
BERT-CNN	Bidirectional Encoder-Decoder Transformer Convolutional Neural Network
BiGRU	Bidirectional Gated Recurrent Neural Networks
Bi-LSTM	Bidirectional Long Short-Term Memory
BPMN	Business Process Model and Notation
CNN	Convolutional Neural Network
CrowdRE	Crowd-Based Requirements Engineering
DF4RT	Deep Forest for Requirements Traceability
DL	Deep Learning

DT	Decision Tree
FPDM	Fault-Prone Software Requirements Specification Detection Model
FR	Functional Requirement
GDPR	General Data Protection Regulation
GNB	Gaussian Naïve Bayes
GPT	Generative Pre-trained Transformer
GRU	Gated Recurrent Unit
IE	Information Extraction
KNN	K-Nearest Neighbors
LLM	Large Language Model
LR	Linear Regression
LogR	Logistic Regression
LSTM	Long Short-Term Memory
ML	Machine Learning
MLP	Multilayer Perceptron
MNB	Multinomial Naïve Bayes
MTBE	Model Transformation by-Example
NB	Naïve Bayes
NER	Name Entity Recognition
NFR	Non-Functional Requirement
NL	Natural Language
NLP	Natural Language Processing
NLP4RE	Natural Language Processing for Requirements Engineering
NLP4ReF	Natural Language Processing for Requirements Forecasting
NN	Neural Networks
PLM	Pre-trained Language Models
PMBOK	Project Management Body of Knowledge
PoS	Part-of-speech (PoS tagging)
RE	Requirements Engineering
READ	Requirement Engineering Analysis Design
ReqVec	Semantic vector representation for requirements
RF	Random Forest
RNN	Recurrent Neural Network
ROF	Rule-Based Ontology Framework
SDLC	Software development life-cycle
SRS	Software Requirements Specification
SRXCRM	System Requirement eXtraction with Chunking and Rule Mining
SVM	Support Vector Machines
SyAcUcNER	System Actor Use-Case Named Entity Recognizer
TF-IDF	Term frequency-inverse document frequency
TLR-ELtoR	Evolutionary Learning to Rank for Traceability Link Recovery
T5	Text-To-Text Transfer Transformer
UML	Unified Modeling Language

References

1. Liubchenko, V. The Machine Learning Techniques for Enhancing Software Requirement Specification: Literature Review. In Proceedings of the 2023 IEEE 12th International Conference on Intelligent Data Acquisition and Advanced Computing Systems: Technology and Applications (IDAACS), Dortmund, Germany, 7–9 September 2023; Volume 1, pp. 10–14. [CrossRef]
2. Jiang, Y.; Li, X.; Luo, H.; Yin, S.; Kaynak, O. Quo vadis artificial intelligence? *Discov. Artif. Intell.* **2022**, *2*, 4. [CrossRef]
3. Min, B.; Ross, H.; Sulem, E.; Veyseh, A.P.B.; Nguyen, T.H.; Sainz, O.; Agirre, E.; Heintz, I.; Roth, D. Recent Advances in Natural Language Processing via Large Pre-trained Language Models: A Survey. *ACM Comput. Surv.* **2023**, *56*, 1–40. [CrossRef]
4. Fernandes, J.M.; Machado, R.J. *Requirements in Engineering Projects*; Springer: Cham, Switzerland, 2016.
5. Pressman, R. *Software Engineering: A Practitioner's Approach*; McGraw-Hill Higher Education, McGraw-Hill Education: New York, NY, USA, 2010.

6. van Lamsweerde, A. *Requirements Engineering: From System Goals to UML Models to Software Specifications*; Wiley: Hoboken, NJ, USA, 2009.
7. Zamani, K.; Zowghi, D.; Arora, C. Machine Learning in Requirements Engineering: A Mapping Study. In Proceedings of the 2021 IEEE 29th International Requirements Engineering Conference Workshops (REW), Notre Dame, IN, USA, 20–24 September 2021; pp. 116–125. [CrossRef]
8. Liu, K.; Reddivari, S.; Reddivari, K. Artificial Intelligence in Software Requirements Engineering: State-of-the-Art. In Proceedings of the 2022 IEEE 23rd International Conference on Information Reuse and Integration for Data Science (IRI), San Diego, CA, USA, 9–11 August 2022; pp. 106–111. [CrossRef]
9. Navaei, M.; Tabrizi, N. Machine Learning in Software Development Life Cycle: A Comprehensive Review. *ENASE* **2022**, *1*, 344–354.
10. Abdelnabi, E.A.; Maatuk, A.M.; Hagal, M. Generating UML Class Diagram from Natural Language Requirements: A Survey of Approaches and Techniques. In Proceedings of the 2021 IEEE 1st International Maghreb Meeting of the Conference on Sciences and Techniques of Automatic Control and Computer Engineering MI-STA, Tripoli, Libya, 25–27 May 2021; pp. 288–293. [CrossRef]
11. Kulkarni, V.; Kolhe, A.; Kulkarni, J. Intelligent Software Engineering: The Significance of Artificial Intelligence Techniques in Enhancing Software Development Lifecycle Processes. In *Proceedings of the Intelligent Systems Design and Applications*; Abraham, A., Gandhi, N., Hanne, T., Hong, T.P., Nogueira Rios, T., Ding, W., Eds.; Springer: Cham, Switzerland, 2022; pp. 67–82.
12. Sofian, H.; Yunus, N.A.M.; Ahmad, R. Systematic Mapping: Artificial Intelligence Techniques in Software Engineering. *IEEE Access* **2022**, *10*, 51021–51040. [CrossRef]
13. Sufian, M.; Khan, Z.; Rehman, S.; Haider Butt, W. A Systematic Literature Review: Software Requirements Prioritization Techniques. In Proceedings of the 2018 International Conference on Frontiers of Information Technology (FIT), Islamabad, Pakistan, 17–19 December 2018; pp. 35–40. [CrossRef]
14. Talele, P.; Phalnikar, R. Classification and Prioritisation of Software Requirements using Machine Learning—A Systematic Review. In Proceedings of the 2021 11th International Conference on Cloud Computing, Data Science & Engineering (Confluence), Noida, India, 28–29 January 2021; pp. 912–918. [CrossRef]
15. Kaur, K.; Kaur, P. The application of AI techniques in requirements classification: A systematic mapping. *Artif. Intell. Rev.* **2024**, *57*, 57. [CrossRef]
16. López-Hernández, D.A.; Octavio Ocharán-Hernández, J.; Mezura-Montes, E.; Sánchez-García, A. Automatic Classification of Software Requirements using Artificial Neural Networks: A Systematic Literature Review. In Proceedings of the 2021 9th International Conference in Software Engineering Research and Innovation (CONISOFT), San Diego, CA, USA, 25–29 October 2021; pp. 152–160. [CrossRef]
17. Pérez-Verdejo, J.M.; Sánchez-García, A.J.; Ocharán-Hernández, J.O. A Systematic Literature Review on Machine Learning for Automated Requirements Classification. In Proceedings of the 2020 8th International Conference in Software Engineering Research and Innovation (CONISOFT), Chetumal, Mexico, 4–6 November 2020; pp. 21–28. [CrossRef]
18. Li, X.; Wang, B.; Wan, H.; Deng, Y.; Wang, Z. Applications of Machine Learning in Requirements Traceability: A Systematic Mapping Study. In Proceedings of the 35th International Conference on Software Engineering and Knowledge Engineering, Virtual, 5–10 July 2023; pp. 566–571. [CrossRef]
19. Yadav, A.; Patel, A.; Shah, M. A comprehensive review on resolving ambiguities in natural language processing. *AI Open* **2021**, *2*, 85–92. [CrossRef]
20. Aberkane, A.J.; Poels, G.; Broucke, S.V. Exploring Automated GDPR-Compliance in Requirements Engineering: A Systematic Mapping Study. *IEEE Access* **2021**, *9*, 66542–66559. [CrossRef]
21. Page, M.J.; McKenzie, J.E.; Bossuyt, P.M.; Boutron, I.; Hoffmann, T.C.; Mulrow, C.D.; Shamseer, L.; Tetzlaff, J.M.; Akl, E.A.; Brennan, S.E.; et al. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *BMJ* **2021**, *372*, n71. Available online: https://www.bmj.com/content/372/bmj.n71.full.pdf (accessed on 9 May 2024). [CrossRef]
22. Nagarhalli, T.P.; Vaze, V.; Rana, N.K. Impact of Machine Learning in Natural Language Processing: A Review. In Proceedings of the 2021 Third International Conference on Intelligent Communication Technologies and Virtual Mobile Networks (ICICV), Tirunelveli, India, 4–6 February 2021; pp. 1529–1534. [CrossRef]
23. Badillo, S.; Banfai, B.; Birzele, F.; Davydov, I.I.; Hutchinson, L.; Kam-Thong, T.; Siebourg-Polster, J.; Steiert, B.; Zhang, J.D. An Introduction to Machine Learning. *Clin. Pharmacol. Ther.* **2020**, *107*, 871–885. [CrossRef]
24. Pouyanfar, S.; Sadiq, S.; Yan, Y.; Tian, H.; Tao, Y.; Reyes, M.P.; Shyu, M.L.; Chen, S.C.; Iyengar, S.S. A Survey on Deep Learning: Algorithms, Techniques, and Applications. *ACM Comput. Surv.* **2018**, *51*. [CrossRef]
25. Zhao, L.; Alhoshan, W.; Ferrari, A.; Letsholo, K.J. Classification of natural language processing techniques for requirements engineering. *arXiv* **2022**, arXiv:2204.04282.
26. Zhang, H.; Shafiq, M.O. Survey of transformers and towards ensemble learning using transformers for natural language processing. *J. Big Data* **2024**, *11*, 25. [CrossRef]

27. da Cruz Mello., O.; Fontoura, L.M. Challenges in Requirements Engineering and Its Solutions: A Systematic Review. In *Proceedings of the 24th International Conference on Enterprise Information Systems—Volume 2: ICEIS*; INSTICC, SciTePress: Setúbal, Portugal, 2022; pp. 70–77. [CrossRef]
28. Ahmed, S.; Ahmed, A.; Eisty, N.U. Automatic Transformation of Natural to Unified Modeling Language: A Systematic Review. In Proceedings of the 2022 IEEE/ACIS 20th International Conference on Software Engineering Research, Management and Applications (SERA), Las Vegas, NV, USA, 25–27 May 2022; pp. 112–119. [CrossRef]
29. Sonbol, R.; Rebdawi, G.; Ghneim, N. The Use of NLP-Based Text Representation Techniques to Support Requirement Engineering Tasks: A Systematic Mapping Review. *IEEE Access* **2022**, *10*, 62811–62830. [CrossRef]
30. Atoum, I.; Baklizi, M.K.; Alsmadi, I.; Otoom, A.A.; Alhersh, T.; Ababneh, J.; Almalki, J.; Alshahrani, S.M. Challenges of Software Requirements Quality Assurance and Validation: A Systematic Literature Review. *IEEE Access* **2021**, *9*, 137613–137634. [CrossRef]
31. Santos, R.; Groen, E.C.; Villela, K. An Overview of User Feedback Classification Approaches. In Proceedings of the REFSQ Workshops, Essen, Germany, 18–21 March 2019; Volume 3; pp. 357–369.
32. Ahmad, A.; Feng, C.; Tahir, A.; Khan, A.; Waqas, M.; Ahmad, S.; Ullah, A. An Empirical Evaluation of Machine Learning Algorithms for Identifying Software Requirements on Stack Overflow: Initial Results. In Proceedings of the 2019 IEEE 10th International Conference on Software Engineering and Service Science (ICSESS), Beijing, China, 18–20 October 2019; pp. 689–693. [CrossRef]
33. Budake, R.; Bhoite, S.; Kharade, K. Identification and Classification of Functional and Nonfunctional Software Requirements Using Machine Learning. *AIP Conf. Proc.* **2023**, *2946*, 050009. [CrossRef]
34. Talele, P.; Phalnikar, R. Multiple correlation based decision tree model for classification of software requirements. *Int. J. Comput. Sci. Eng.* **2023**, *26*, 131502. [CrossRef]
35. Alhoshan, W.; Ferrari, A.; Zhao, L. Zero-shot learning for requirements classification: An exploratory study. *Inf. Softw. Technol.* **2023**, *159*, 107202. [CrossRef]
36. Talele, P.; Phalnikar, R.; Apte, S.; Talele, H. Semi-automated Software Requirements Categorisation using Machine Learning Algorithms. *Int. J. Electr. Comput. Eng. Syst.* **2023**, *14*, 1107–1114. [CrossRef]
37. Tasnim, A.; Akhter, N.; Ali, K. A Fine Tuned Ensemble Approach to Classify Requirement from User Story. In Proceedings of the 2023 26th International Conference on Computer and Information Technology, ICCIT 2023, Cox's Bazar, Bangladesh, 13–15 December 2023. [CrossRef]
38. Kaur, K.; Kaur, P. SABDM: A self-attention based bidirectional-RNN deep model for requirements classification. *J. Softw. Evol. Process* **2024**, *36*. [CrossRef]
39. Peer, J.; Mordecai, Y.; Reich, Y. NLP4ReF: Requirements Classification and Forecasting: From Model-Based Design to Large Language Models. In Proceedings of the IEEE Aerospace Conference Proceedings, Big Sky, MT, USA, 2–9 March 2024. [CrossRef]
40. Budake, R.; Bhoite, S.; Kharade, K. Machine Learning-Based Identification as Well as Classification of Functional and Non-functional Requirements. In *High Performance Computing, Smart Devices and Networks*; Springer: Singapore, 2024; Volume 1087. [CrossRef]
41. AlDhafer, O.; Ahmad, I.; Mahmood, S. An end-to-end Deep Learning system for requirements classification using recurrent neural networks. *Inf. Softw. Technol.* **2022**, *147*, 106877. [CrossRef]
42. Patel, V.; Mehta, P.; Lavingia, K. Software Requirement Classification Using Machine Learning Algorithms. In Proceedings of the 2023 International Conference on Artificial Intelligence and Applications, ICAIA 2023 and Alliance Technology Conference, ATCON-1 2023—Proceeding, Bangalore, India, 21–22 April 2023. [CrossRef]
43. Jp, S.; Menon, V.; Soman, K.; Ojha, A. A Non-Exclusive Multi-Class Convolutional Neural Network for the Classification of Functional Requirements in AUTOSAR Software Requirement Specification Text. *IEEE Access* **2022**, *10*, 117707–117714. [CrossRef]
44. Nayak, U.A.; Swarnalatha, K.; Balachandra, A. Feasibility Study of Machine Learning & AI Algorithms for Classifying Software Requirements. In Proceedings of the MysuruCon 2022—2022 IEEE 2nd Mysore Sub Section International Conference, Mysuru, India, 16–17 October 2022. [CrossRef]
45. Kaur, K.; Kaur, P. BERT-CNN: Improving BERT for Requirements Classification using CNN. *Procedia Comput. Sci.* **2022**, *218*, 2604–2611. [CrossRef]
46. Vijayvargiya, S.; Kumar, L.; Murthy, L.; Misra, S. Software Requirements Classification using Deep Learning Approach with Various Hidden Layers. In Proceedings of the 17th Conference on Computer Science and Intelligence Systems, FedCSIS 2022, Sofia, Bulgaria, 4–7 September 2022. [CrossRef]
47. Luo, X.; Xue, Y.; Xing, Z.; Sun, J. PRCBERT: Prompt Learning for Requirement Classification using BERT-based Pretrained Language Models. In Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering, Rochester, MI, USA, 10–14 October 2022. [CrossRef]
48. Magalhães, C.; Araujo, J.; Sardinha, A. MARE: An Active Learning Approach for Requirements Classification. In Proceedings of the IEEE 29th International Requirements Engineering Conference (RE), Notre Dame, IN, USA, 20–24 September 2021. [CrossRef]

49. Pérez-Verdejo, J.; Sánchez-García, Á.; Ocharán-Hernández, J.; Mezura-Montes, E.; Cortés-Verdín, K. Requirements and GitHub Issues: An Automated Approach for Quality Requirements Classification. *Program. Comput. Softw.* **2021**, *47*, 704–721. [CrossRef]
50. Quba, G.; Qaisi, H.A.; Althunibat, A.; Alzu'Bi, S. Software Requirements Classification using Machine Learning algorithm's. In Proceedings of the 2021 International Conference on Information Technology, ICIT 2021—Proceedings, Amman, Jordan, 14–15 July 2021. [CrossRef]
51. Dave, D.; Anu, V. Identifying Functional and Non-functional Software Requirements from User App Reviews. In Proceedings of the 2022 IEEE International IOT, Electronics and Mechatronics Conference, IEMTRONICS 2022, Toronto, ON, Canada, 1–4 June 2022. [CrossRef]
52. Dave, D.; Anu, V.; Varde, A. Automating the Classification of Requirements Data. In Proceedings of the IEEE International Conference on Big Data, Big Data 2021, Orlando, FL, USA, 15–18 December 2021. [CrossRef]
53. Vijayvargiya, S.; Kumar, L.; Malapati, A.; Murthy, L.; Misra, S. Software Functional Requirements Classification Using Ensemble Learning. In Proceedings of the International Conference on Computational Science and Its Applications, Malaga, Spain, 4–7 July 2022; Lecture Notes in Computer Science (Including Subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), LNCS; Springer: Cham, Switzerland, 2022; Volume 13381. [CrossRef]
54. Rahimi, N.; Eassa, F.; Elrefaei, L. An ensemble Machine Learning technique for functional requirement classification. *Symmetry* **2020**, *12*, 1601. [CrossRef]
55. Panichella, S.; Ruiz, M. Requirements-Collector: Automating requirements specification from elicitation sessions and user feedback. In Proceedings of the IEEE International Conference on Requirements Engineering, Zurich, Switzerland, 31 August–4 September 2020. [CrossRef]
56. Surana, C.S.R.K.; Gupta, D.B.; Shankar, S.P. Intelligent Chatbot for Requirements Elicitation and Classification. In Proceedings of the 2019 4th IEEE International Conference on Recent Trends on Electronics, Information, Communication and Technology, RTEICT 2019—Proceedings, Bangalore, India, 17–18 May 2019. [CrossRef]
57. Li, L.; Jin-An, N.; Kasirun, Z.; Piaw, C. An empirical comparison of Machine Learning algorithms for classification of software requirements. *Int. J. Adv. Comput. Sci. Appl.* **2019**, *10*. [CrossRef]
58. Kim, D.K.; Chen, J.; Ming, H.; Lu, L. Assessment of ChatGPT's Proficiency in Software Development. In Proceedings of the Congress in Computer Science, Computer Engineering, and Applied Computing, Las Vegas, NV, USA, 24–27 July 2023. [CrossRef]
59. Apte, S.; Honrao, Y.; Shinde, R.; Talele, P.; Phalnikar, R. Automatic Extraction of Software Requirements Using Machine Learning. In *International Conference on Information and Communication Technology for Intelligent Systems*; LNNS; Springer: Singapore, 2023; Volume 719. [CrossRef]
60. Chatterjee, R.; Ahmed, A.; Anish, P. Identification and Classification of Architecturally Significant Functional Requirements. In Proceedings of the 7th International Workshop on Artificial Intelligence and Requirements Engineering, AIRE 2020, Zurich, Switzerland, 1 September 2020. [CrossRef]
61. Baker, C.; Deng, L.; Chakraborty, S.; Dehlinger, J. Automatic multi-class non-functional software requirements classification using neural networks. In Proceedings of the International Computer Software and Applications Conference, Milwaukee, WI, USA, 15–19 July 2019; Volume 2. [CrossRef]
62. Lafi, M.; Abdelqader, A. Automated Business Rules Classification Using Machine Learning to Enhance Software Requirements Elicitation. In Proceedings of the 2023 International Conference on Information Technology: Cybersecurity Challenges for Sustainable Cities, ICIT 2023-Proceeding, Amman, Jordan, 9–10 August 2023. [CrossRef]
63. Gobov, D.; Huchenko, I. Software requirements elicitation techniques selection method for the project scope management. In Proceedings of the CEUR Workshop Proceedings, Online, 13–15 December 2021; Volume 2851.
64. Yeow, J.; Rana, M.; Majid, N.A. An Automated Model of Software Requirement Engineering Using GPT-3.5. In Proceedings of the 2024 ASU International Conference in Emerging Technologies for Sustainability and Intelligent Systems, ICETSIS 2024, Manama, Bahrain, 28–29 January 2024. [CrossRef]
65. Tizard, J.; Devine, P.; Wang, H.; Blincoe, K. A Software Requirements Ecosystem: Linking Forum, Issue Tracker, and FAQs for Requirements Management. *IEEE Trans. Softw. Eng.* **2023**, *49*, 2381–2393. [CrossRef]
66. Shen, Y.; Breaux, T. Stakeholder Preference Extraction from Scenarios. *IEEE Trans. Softw. Eng.* **2024**, *50*, 69–84. [CrossRef]
67. Devine, P.; Koh, Y.; Blincoe, K. Evaluating software user feedback classifier performance on unseen apps, datasets, and metadata. *Empir. Softw. Eng.* **2023**, *28*, 26. [CrossRef]
68. Gudaparthi, H.; Niu, N.; Wang, B.; Bhowmik, T.; Liu, H.; Zhang, J.; Savolainen, J.; Horton, G.; Crowe, S.; Scherz, T.; et al. Prompting Creative Requirements via Traceable and Adversarial Examples in Deep Learning. In Proceedings of the IEEE 31st International Requirements Engineering Conference (RE), Hannover, Germany, 4–8 September 2023. [CrossRef]
69. Gôlo, M.; Araújo, A.; Rossi, R.; Marcacini, R. Detecting relevant app reviews for software evolution and maintenance through multimodal one-class learning. *Inf. Softw. Technol.* **2022**, *151*, 106998. [CrossRef]

70. Kauschinger, M.; Vieth, N.; Schreieck, M.; Krcmar, H. Detecting Feature Requests of Third-Party Developers through Machine Learning: A Case Study of the SAP Community. In Proceedings of the 56th Annual Hawaii International Conference on System Sciences, Kaanapali Beach, Maui, Hawaii, USA, 3–6 January 2023.
71. Mehder, S.; Aydemir, F.B. Classification of Issue Discussions in Open Source Projects Using Deep Language Models. In Proceedings of the IEEE International Conference on Requirements Engineering, Melbourne, Australia, 15–19 August 2022. [CrossRef]
72. Devine, P. Finding Appropriate User Feedback Analysis Techniques for Multiple Data Domains. In Proceedings of the International Conference on Software Engineering, Pittsburgh, PA, USA, 21–29 May 2022. [CrossRef]
73. Araujo, A.; Marcacini, R. Hierarchical Cluster Labeling of Software Requirements using Contextual Word Embeddings. In Proceedings of the ACM International Conference Proceeding Series, Joinville, Brazil, 27 September–1 October 2021. [CrossRef]
74. Bhatia, K.; Sharma, A. Sector classification for crowd-based software requirements. In Proceedings of the ACM Symposium on Applied Computing, Virtual Event, Republic of Korea, 22–26 March 2021. [CrossRef]
75. Do, Q.; Bhowmik, T.; Bradshaw, G. Capturing creative requirements via requirements reuse: A machine learning-based approach. *J. Syst. Softw.* **2020**, *170*, 110730. [CrossRef]
76. Iqbal, T.; Seyff, N.; Mendez, D. Generating requirements out of thin air: Towards automated feature identification for new apps. In Proceedings of the IEEE 27th International Requirements Engineering Conference Workshops, REW 2019, Jeju, Republic of Korea, 23–27 September 2019. [CrossRef]
77. Morales-Ramirez, I.; Kifetew, F.; Perini, A. Speech-acts based analysis for requirements discovery from online discussions. *Inf. Syst.* **2019**, *86*, 94–112. [CrossRef]
78. Do, Q.; Chekuri, S.; Bhowmik, T. Automated Support to Capture Creative Requirements via Requirements Reuse. In *Reuse in the Big Data Era, Proceedings of the 18th International Conference on Software and Systems Reuse, ICSR 2019, Cincinnati, OH, USA, 26–28 June 2019*; Springer: Berlin/Heidelberg, Germany, 2019; Volume 11602 LNCS. [CrossRef]
79. Tizard, J.; Wang, H.; Yohannes, L.; Blincoe, K. Can a conversation paint a picture? Mining requirements in software forums. In Proceedings of the IEEE International Conference on Requirements Engineering, Jeju, Republic of Korea, 23–27 September 2019. [CrossRef]
80. Peng, S.; Xu, L.; Jiang, W. Itemization Framework of Requirements using Machine Reading Comprehension. In Proceedings of the SPIE—The International Society for Optical Engineering, Chongqing, China, 29–31 July 2022; Volume 12451. [CrossRef]
81. Ehresmann, M.; Beyer, J.; Fasoulas, S.; Schorfmann, M.; Brudna, T.; Schoolmann, I.; Brüggmann, J.; Hönle, A.; Gerlich, R.; Gerlich, R. ExANT: Exploring NLP AI Systems for Requirements Development. In Proceedings of the International Astronautical Congress, IAC, Baku, Azerbaijan, 2–6 October 2023.
82. Muhamad, F.; Hamid, S.A.; Subramaniam, H.; Rashid, R.A.; Fahmi, F. Fault-Prone Software Requirements Specification Detection Using Ensemble Learning for Edge/Cloud Applications. *Appl. Sci.* **2023**, *13*, 8368. [CrossRef]
83. Nguyen, T.; Sayar, I.; Ebersold, S.; Bruel, J.M. Identifying and fixing ambiguities in, and semantically accurate formalisation of, behavioural requirements. *Softw. Syst. Model.* **2024**, *23*, 1513–1545. [CrossRef]
84. Berhanu, F.; Alemneh, E. Classification and Prioritization of Requirements Smells Using Machine Learning Techniques. In Proceedings of the 2023 International Conference on Information and Communication Technology for Development for Africa, ICT4DA 2023, Bahir Dar, Ethiopia, 26–28 October 2023. [CrossRef]
85. Luitel, D.; Hassani, S.; Sabetzadeh, M. Improving requirements completeness: Automated assistance through large language models. *Requir. Eng.* **2024**, *29*, 73–95. [CrossRef]
86. Habib, M.; Wagner, S.; Graziotin, D. Detecting Requirements Smells with Deep Learning: Experiences, Challenges and Future Work. In Proceedings of the IEEE International Conference on Requirements Engineering, Notre Dame, IN, USA, 20–24 September 2021. [CrossRef]
87. Liu, C.; Zhao, Z.; Zhang, L.; Li, Z. Automated conditional statements checking for complete natural language requirements specification. *Appl. Sci.* **2021**, *11*, 7892. [CrossRef]
88. Singh, S.; Saikia, L.P.; Baruah, S. A study on Quality Assessment of Requirement Engineering Document using Text Classification Technique. In Proceedings of the 2nd International Conference on Electronics and Sustainable Communication Systems, ICESC 2021, Coimbatore, India, 4–6 August 2021. [CrossRef]
89. Moharil, A.; Sharma, A. Identification of Intra-Domain Ambiguity using Transformer-based Machine Learning. In Proceedings of the 1st International Workshop on Natural Language-Based Software Engineering, NLBSE 2022, Pittsburgh, PA, USA, 21 May 2022. [CrossRef]
90. Subedi, I.; Singh, M.; Ramasamy, V.; Walia, G. Application of back-translation: A transfer learning approach to identify ambiguous software requirements. In Proceedings of the ACMSE Conference—ACMSE 2021: The Annual ACM Southeast Conference, Virtual Event, USA, 15–17 April 2021. [CrossRef]

91. Onyeka, E.; Varde, A.; Anu, V.; Tandon, N.; Daramola, O. Using Commonsense Knowledge and Text Mining for Implicit Requirements Localization. In Proceedings of the International Conference on Tools with Artificial Intelligence, ICTAI, Baltimore, MD, USA, 9–11 November 2020. [CrossRef]
92. Femmer, H.; Müller, A.; Eder, S. Semantic Similarities in Natural Language Requirements. In *Software Quality: Quality Intelligence in Software and Systems Engineering, Proceedings of the 12th International Conference, SWQD 2020, Vienna, Austria, 14–17 January 2020*; LNBIP; Springer International Publishing: Cham, Switzerland; Volume 371. [CrossRef]
93. Memon, K.; Xia, X. Deciphering and analyzing software requirements employing the techniques of Natural Language processing. In Proceedings of the ACM International Conference on Mathematics and Artificial Intelligence, Chegndu, China, 12–15 April 2019. [CrossRef]
94. Atoum, I. Measurement of key performance indicators of user experience based on software requirements. *Sci. Comput. Program.* **2023**, *226*, 102929. [CrossRef]
95. Althar, R.; Samanta, D. BERT-Based Secure and Smart Management System for Processing Software Development Requirements from Security Perspective. In *Machine Intelligence and Data Science Applications*; Springer: Singapore, 2022; Volume 132. [CrossRef]
96. Imtiaz, S.; Amin, M.; Do, A.; Iannucci, S.; Bhowmik, T. Predicting Vulnerability for Requirements. In Proceedings of the IEEE 22nd International Conference on Information Reuse and Integration for Data Science, IRI 2021, Las Vegas, NV, USA, 10–12 August 2021. [CrossRef]
97. Aberkane, A.-J. Automated GDPR-compliance in requirements engineering. In Proceedings of the Doctoral Consortium Papers Presented at the 33rd International Conference on Advanced Information Systems Engineering (CAiSE 2021), Virtual Event, Australia, Victoria, 28 June–2 July 2021; Volume 2906. Available online: https://ceur-ws.org/Vol-2906/paper3.pdf (accessed on 22 June 2024).
98. dos Santos, R.; Villela, K.; Avila, D.; Thom, L. A practical user feedback classifier for software quality characteristics. In Proceedings of the International Conference on Software Engineering and Knowledge Engineering, SEKE, Virtual conference at the KSIR Virtual Conference Center, Pittsburgh, USA, 1–10 July 2021. [CrossRef]
99. Atoum, I.; Almalki, J.; Alshahrani, S.; Shehri, W. Towards Measuring User Experience based on Software Requirements. *Int. J. Adv. Comput. Sci. Appl.* **2021**, *12*. [CrossRef]
100. Hovorushchenko, T.; Pavlova, O. Method of activity of ontology-based intelligent agent for evaluating initial stages of the software lifecycle. In Proceedings of the Advances in Intelligent Systems and Computing, Kyiv, Ukraine, 4–7 June 2018; Springer: Cham, Switzerland, 2019; Volume 836. [CrossRef]
101. Groher, I.; Seyff, N.; Iqbal, T. Towards automatically identifying potential sustainability effects of requirements. In Proceedings of the CEUR Workshop Proceedings, Castiglione della Pescaia (Grosseto), Italy, 16–19 June 2019; Volume 2541.
102. Chazette, L. Mitigating challenges in the elicitation and analysis of transparency requirements. In Proceedings of the IEEE International Conference on Requirements Engineering, Jeju, Korea, 23–27 September 2019. [CrossRef]
103. Adithya, V.; Deepak, G. OntoReq: An Ontology Focused Collective Knowledge Approach for Requirement Traceability Modelling. In *AI Systems and the Internet of Things in the Digital Era, Proceedings of EAMMIS 2021, Istanbul, Turkey, 19–20 March 2021*; Musleh Al-Sartawi, A.M., Razzaque, A., Kamal, M.M., Eds.; Springer International Publishing: Cham, Switzerland, 2021; pp. 358–370. [CrossRef]
104. Fadhlurrohman, D.; Sabariah, M.; Alibasa, M.; Husen, J. Naïve Bayes Classification Model for Precondition-Postcondition in Software Requirements. In Proceedings of the 2023 International Conference on Data Science and Its Applications, ICoDSA 2023, Bandung, Indonesia, 9–10 August 2023. [CrossRef]
105. Sawada, K.; Pomerantz, M.; Razo, G.; Clark, M. Intelligent requirement-to-test case traceability system via Natural Language Processing and Machine Learning. In Proceedings of the IEEE 9th International Conference on Space Mission Challenges for Information Technology, SMC-IT 2023, Pasadena, CA, USA, 18–27 July 2023. [CrossRef]
106. Subedi, I.; Singh, M.; Ramasamy, V.; Walia, G. Classification of Testable and Valuable User Stories by using Supervised Machine Learning Classifiers. In Proceedings of the IEEE International Symposium on Software Reliability Engineering Workshops, ISSREW 2021, Wuhan, China, 25–28 October 2021. [CrossRef]
107. Merugu, R.; Chinnam, S. Automated cloud service based quality requirement classification for software requirement specification. *Evol. Intell.* **2021**, *14*, 389–394. [CrossRef]
108. Ueda, K.; Tsukada, H. Accuracy Improvement by Training Data Selection in Automatic Test Cases Generation Method. In Proceedings of the 2021 9th International Conference on Information and Education Technology, ICIET 2021, Okayama, Japan, 27–29 March 2021. [CrossRef]
109. Kikuma, K.; Yamada, T.; Sato, K.; Ueda, K. Preparation method in automated test case generation using machine learning. In Proceedings of the ACM International Conference Proceeding Series, Hanoi Ha Long Bay, Vietnam, 4–6 December 2019. [CrossRef]
110. Kaya, A.; Keceli, A.; Catal, C.; Tekinerdogan, B. Model analytics for defect prediction based on design-level metrics and sampling techniques. In *Model Management and Analytics for Large Scale Systems*; Academic Press: Cambridge, MA, USA, 2019. [CrossRef]

111. Petcuşin, F.; Stănescu, L.; Bădică, C. An Experiment on Automated Requirements Mapping Using Deep Learning Methods. In *Proceedings of the Studies in Computational Intelligence*; Springer International Publishing: Berlin/Heidelberg, Germany, 2020; Volume 868. [CrossRef]
112. Zhang, J.; Yuan, M.; Huang, Z. Software Requirements Elicitation Based on Ontology Learning. In *Proceedings of the Communications in Computer and Information Science*; Springer: Singapore, 2019; Volume 861. [CrossRef]
113. Lano, K.; Kolahdouz-Rahimi, S.; Fang, S. Model Transformation Development Using Automated Requirements Analysis, Metamodel Matching, and Transformation by Example. *ACM Trans. Softw. Eng. Methodol.* **2022**, *31*, 3471907. [CrossRef]
114. Koscinski, V.; Gambardella, C.; Gerstner, E.; Zappavigna, M.; Cassetti, J.; Mirakhorli, M. A Natural Language Processing Technique for Formalization of Systems Requirement Specifications. In Proceedings of the IEEE International Conference on Requirements Engineering, Notre Dame, IN, USA, 20–24 September 2021. [CrossRef]
115. Yanuarifiani, A.; Chua, F.F.; Chan, G.Y. Feasibility Analysis of a Rule-Based Ontology Framework (ROF) for Auto-Generation of Requirements Specification. In Proceedings of the IEEE International Conference on Artificial Intelligence in Engineering and Technology, IICAIET 2020, Kota Kinabalu, Malaysia, 26–27 September 2020. [CrossRef]
116. Arulmohan, S.; Meurs, M.J.; Mosser, S. Extracting Domain Models from Textual Requirements in the Era of Large Language Models. In Proceedings of the ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion, MODELS-C 2023, Västerås, Sweden, 1–6 October 2023. [CrossRef]
117. Saini, R.; Mussbacher, G.; Guo, J.; Kienzle, J. Automated, interactive, and traceable domain modelling empowered by artificial intelligence. *Softw. Syst. Model.* **2022**, *21*, 1015–1045. [CrossRef]
118. Zhang, J.; Chen, S.; Hua, J.; Niu, N.; Liu, C. Automatic Terminology Extraction and Ranking for Feature Modeling. In Proceedings of the IEEE International Requirements Engineering Conference (RE), Melbourne, Australia, 15–19 August 2022. [CrossRef]
119. Sree-Kumar, A.; Planas, E.; Clarisó, R. Validating Feature Models with Respect to Textual Product Line Specifications. In Proceedings of the ACM International Conference Proceeding Series, Krems, Austria, 9–11 February 2021. [CrossRef]
120. Bonner, M.; Zeller, M.; Schulz, G.; Beyer, D.; Olteanu, M. Automated Traceability between Requirements and Model-Based Design. In Proceedings of the REFSQ Posters and Tools (CEUR Workshop Proceedings), Barcelona, Spain, 17–20 April 2023; Volume 3378.
121. Bashir, N.; Bilal, M.; Liaqat, M.; Marjani, M.; Malik, N.; Ali, M. Modeling Class Diagram using NLP in Object-Oriented Designing. In Proceedings of the IEEE 4th National Computing Colleges Conference, NCCC 2021, Taif, Saudi Arabia, 27–28 March 2021. [CrossRef]
122. Vineetha, V.; Samuel, P. A Multinomial Naïve Bayes Classifier for identifying Actors and Use Cases from Software Requirement Specification documents. In Proceedings of the 2nd International Conference on Intelligent Technologies, CONIT 2022, Hubli, India, 24–26 June 2022. [CrossRef]
123. Schouten, M.; Ramackers, G.; Verberne, S. Preprocessing Requirements Documents for Automatic UML Modelling. In *Proceedings of the Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*; LNCS; Springer: Cham, Switzerland, 2022; Volume 13286. [CrossRef]
124. Imam, A.; Alhroob, A.; Alzyadat, W. SVM Machine Learning Classifier to Automate the Extraction of SRS Elements. *Int. J. Adv. Comput. Sci. Appl.* **2021**, *12*, 0120322. [CrossRef]
125. Leitão, V.; Medeiros, I. SRXCRM: Discovering Association Rules between System Requirements and Product Specifications. In Proceedings of the REFSQ Workshops, Essen, Germany, 12–15 April 2021; Volume 2857. Available online: https://ceur-ws.org/Vol-2857/nlp4re9.pdf (accessed on 22 June 2024).
126. Sonbol, R.; Rebdawi, G.; Ghneim, N. Towards a Semantic Representation for Functional Software Requirements. In Proceedings of the 7th International Workshop on Artificial Intelligence and Requirements Engineering, AIRE 2020, Zurich, Switzerland, 1 September 2020. [CrossRef]
127. Saini, R.; Mussbacher, G.; Guo, J.; Kienzle, J. Towards Queryable and Traceable Domain Models. In Proceedings of the IEEE International Conference on Requirements Engineering, Zurich, Switzerland, 31 August–4 September 2020. [CrossRef]
128. Tiwari, S.; Rathore, S.; Sagar, S.; Mirani, Y. Identifying Use Case Elements from Textual Specification: A Preliminary Study. In Proceedings of the IEEE International Conference on Requirements Engineering, Zurich, Switzerland, 31 August–4 September 2020. [CrossRef]
129. Saini, R.; Mussbacher, G.; Guo, J.; Kienzle, J. A Neural Network Based Approach to Domain Modelling Relationships and Patterns Recognition. In Proceedings of the 10th International Model-Driven Requirements Engineering Workshop, MoDRE 2020, Zurich, Switzerland, 31 August 2020. [CrossRef]
130. Saini, R.; Mussbacher, G.; Guo, J.; Kienzle, J. DoMoBOT: A bot for automated and interactive domain modelling. In Proceedings of the 23rd ACM/IEEE International Conference on Model Driven Engineering Languages and Systems, MODELS-C 2020—Companion Proceedings, Virtual Event, Canada, 16–23 October 2020. [CrossRef]
131. Ghosh, S.; Bashar, R.; Mukherjee, P.; Chakraborty, B. Automated generation of e-r diagram from a given text in natural language. In Proceedings of the International Conference on Machine Learning and Data Engineering, iCMLDE 2018, Sydney, NSW, Australia, 3–7 December 2019. [CrossRef]

132. Khan, J. Mining requirements arguments from user forums. In Proceedings of the IEEE International Conference on Requirements Engineering, Jeju, Republic of Korea, 23–27 September 2019. [CrossRef]
133. Osman, M.; Alabwaini, N.; Jaber, T.; Alrawashdeh, T. Generate use case from the requirements written in a natural language using Machine Learning. In Proceedings of the 2019 IEEE Jordan International Joint Conference on Electrical Engineering and Information Technology, JEEIT 2019-Proceedings, Amman, Jordan, 9–11 April 2019. [CrossRef]
134. Wever, M.; Rooijen, L.V.; Hamann, H. Multioracle coevolutionary learning of requirements specifications from examples in on-the-fly markets. *Evol. Comput.* **2019**, *28*, 165–193. [CrossRef] [PubMed]
135. Motger, Q.; Borrull, R.; Palomares, C.; Marco, J. OpenReq-DD: A requirements dependency detection tool. In Proceedings of the NLP4RE Workshop, Essen, Germany, 18-21 March, 2019; Volume 2376. Available online: https://ceur-ws.org/Vol-2376/NLP4RE19_paper01.pdf (accessed on 22 June 2025).
136. Deshpande, G.; Arora, C.; Ruhe, G. Data-driven elicitation and optimization of dependencies between requirements. In Proceedings of the IEEE International Conference on Requirements Engineering, Jeju, Republic of Korea, 23–27 September 2019. [CrossRef]
137. Atas, M.; Samer, R.; Felfernig, A. Automated Identification of Type-Specific Dependencies between Requirements. In Proceedings of the IEEE/WIC/ACM International Conference on Web Intelligence, WI 2018, Santiago, Chile, 3–6 December 2019. [CrossRef]
138. Deshpande, G. SReYantra: Automated software requirement inter-dependencies elicitation, analysis and learning. In Proceedings of the IEEE/ACM 41st International Conference on Software Engineering: Companion, ICSE-Companion 2019, Montreal, QC, Canada, 25–31 May 2019. [CrossRef]
139. Wang, B.; Deng, Y.; Wan, H.; Li, X. DF4RT: Deep Forest for Requirements Traceability Recovery Between Use Cases and Source Code. In Proceedings of the IEEE International Conference on Systems, Man and Cybernetics, Oahu, HI, USA, 1–14 October 2023. [CrossRef]
140. Al-walidi, N.; Azab, S.; Khamis, A.; Darwish, N. Clustering-based Automated Requirement Trace Retrieval. *Int. J. Adv. Comput. Sci. Appl.* **2022**, *13*, 0131292. [CrossRef]
141. Marcén, A.C.; Lapeña, R.; Pastor, O.; Cetina, C. Traceability Link Recovery between Requirements and Models using an Evolutionary Algorithm Guided by a Learning to Rank Algorithm: Train control and management case. *J. Syst. Softw.* **2020**, *163*, 110519. [CrossRef]
142. Wang, S.; Li, T.; Yang, Z. Exploring Semantics of Software Artifacts to Improve Requirements Traceability Recovery: A Hybrid Approach. In Proceedings of the Asia-Pacific Software Engineering Conference, APSEC, Putrajaya, Malaysia, 2–5 December 2019. [CrossRef]
143. Chen, L.; Wang, D.; Wang, J.; Wang, Q. Enhancing Unsupervised Requirements Traceability with Sequential Semantics. In Proceedings of the Asia-Pacific Software Engineering Conference, APSEC, Putrajaya, Malaysia, 2–5 December 2019. [CrossRef]
144. Talele, P.; Phalnikar, R. Automated Requirement Prioritisation Technique Using an Updated Adam Optimisation Algorithm. *Int. J. Intell. Syst. Appl. Eng.* **2023**, *11*, 1211–1221.
145. Fatima, A.; Fernandes, A.; Egan, D.; Luca, C. Software Requirements Prioritisation Using Machine Learning. In Proceedings of the 15th International Conference on Agents and Artificial Intelligence, ICAART, Lisbon, Portugal, 22–24 February 2023; Volume 3. [CrossRef]
146. Lunarejo, M. Requirements prioritization based on multiple criteria using Artificial Intelligence techniques. In Proceedings of the IEEE International Conference on Requirements Engineering, Notre Dame, IN, USA, 20–24 September 2021. [CrossRef]
147. Limaylla, M.; Condori-Fernandez, N.; Luaces, M. Towards a Semi-Automated Data-Driven Requirements Prioritization Approach for Reducing Stakeholder Participation in SPL Development. *Eng. Proc.* **2021**, *7*, 27. [CrossRef]
148. Reyad, O.; Dukhan, W.; Marghny, M.; Zanaty, E. Genetic K-Means Adaption Algorithm for Clustering Stakeholders in System Requirements. In *Proceedings of the Advances in Intelligent Systems and Computing*; Springer: Berlin/Heidelberg, Germany, 2021; Volume 1339. [CrossRef]
149. Talele, P.; Phalnikar, R. Software Requirements Classification and Prioritisation Using Machine Learning. In *Proceedings of the Lecture Notes in Networks and Systems*; Springer: Singapore, 2021; Volume 141. [CrossRef]
150. Pereira, F.; Neto, G.; Lima, L.D.; Silva, F.; Peres, L. A Tool For Software Requirement Allocation Using Artificial Intelligence Planning. In Proceedings of the IEEE International Conference on Requirements Engineering, Melbourne, Australia, 15–19 August 2022. [CrossRef]
151. Blincoe, K.; Dehghan, A.; Salaou, A.D.; Neal, A.; Linaker, J.; Damian, D. High-level software requirements and iteration changes: A predictive model. *Empir. Softw. Eng.* **2019**, *24*, 1610–1648. [CrossRef]
152. Liu, Y.; Lin, J.; Cleland-Huang, J.; Vierhauser, M.; Guo, J.; Lohar, S. SENET: A Semantic Web for Supporting Automation of Software Engineering Tasks. In Proceedings of the 7th International Workshop on Artificial Intelligence and Requirements Engineering, AIRE 2020, Zurich, Switzerland, 1 September 2020. [CrossRef]
153. Khan, B.; Naseem, R.; Alam, I.; Khan, I.; Alasmary, H.; Rahman, T. Analysis of Tree-Family Machine Learning Techniques for Risk Prediction in Software Requirements. *IEEE Access* **2022**, *10*, 98220–98231. [CrossRef]

154. Zamani, K. A Prediction Model for Software Requirements Change Impact. In Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, ASE 2021, Melbourne, Australia, 15–19 November 2021. [CrossRef]
155. Cherdsakulwong, N.; Suwannasart, T. Impact Analysis of Test Cases for Changing Inputs or Outputs of Functional Requirements. In Proceedings of the 20th IEEE/ACIS International Conference on Software Engineering, Artificial Intelligence, Networking and Parallel/Distributed Computing, SNPD 2019, Toyama, Japan, 8–11 July 2019. [CrossRef]
156. Olson, R.; Bartley, N.; Urbanowicz, R.; Moore, J. Evaluation of a tree-based pipeline optimization tool for automating data science. In Proceedings of the Genetic and Evolutionary Computation GECCO'16, Denver, CO, USA, 20–24 July 2016; pp. 485–492.
157. Aberkane, A.J. Exploring Ethics in Requirements Engineering. Master's Thesis, Utrecht University, Utrecht, The Netherlands, 2018. Available online: https://studenttheses.uu.nl/handle/20.500.12932/30674 (accessed on 21 June 2024).
158. Peterson, B. Ethical Considerations of AI in Software Engineering: Bias, Reliability, and Human Oversight. Unpublished, 2025. Available online: https://www.researchgate.net/publication/390280753 (accessed on 21 June 2024).